



PYTHON™

Dr Cedric YAO

Ph.D. in Computer Science

Enseignant chercheur au

Département informatique de

Table des matières

Chapitre 0 : Installation et utilisation de Python	8
Comment installer Python	8
Installation sous Windows	8
Étape - 1 : Sélectionnez la version de Python à télécharger.....	9
Étape - 2 : Cliquez sur le bouton Installer maintenant	10
Étape - 3 Installation en cours	12
Étape - 4 : Vérification de l'installation de Python	13
Étape - 5 : Ouverture de IDLE	13
Premier programme Python	14
Invite interactive de l'interprète	14
Utilisation d'un fichier de script (programmation en mode script).....	15
Instructions multilignes	18
Avantages et inconvénients du mode script	19
Premiers pas avec PyCharm	19
Installation de PyCharm	19
Premiers programmes sur Pycharm.....	23
Syntaxe de base de Python	25
Indentation et commentaire en Python	25
Commentaires en Python	26
Types de commentaires	27
Identificateurs Python	28
Chapitre 1 : Notion Fondamentale en Python	29
Introduction	29
Variables, Constantes et Littéraux	29
Variables.....	29
Constantes	29

Types de Conversion	30
Chapitre 2 : Les structures de contrôle	34
I. Introduction à Python pour les structures de contrôle	35
II. Les Conditions : if, elif, else	35
II.1. La condition if ... else	35
II.2. Condition if...elif...else	36
III. Les boucles : for et while	38
III.1. La boucle for	38
III.2. La boucle while	38
IV. Les structures imbriquées	41
IV.1. Le if imbriqué	41
IV.2. Les boucles imbriquées	42
IV.3. Points clés à retenir :	45
V. Les instruction Break et continue	47
VI. Conclusion	50
Chapitre 3 : Les Fonctions et Modules	50
I. Introduction	50
II. Définition des Fonctions	50
II.1. Qu'est-ce qu'une fonction	50
Paramètres et Valeurs de Retour	51
III. Portée des Variables : Locale vs Globale	51
Variables locales	51
Variables globales	52
Exemples pratiques avec des Fonctions	52
IV. Modules et Bibliothèques en Python	53
Qu'est-ce qu'un module ?	53
Utilisation de la bibliothèque math	53

Utilisation de la bibliothèque random	53
V. Création et Importation de Modules Personnalisés	54
Comment créer un module ?	54
Comment importer un module ?	54
Exemple concret : Module pour résoudre des équations mathématiques simples	54
Exercices Pratiques	55
VI. Conclusion	56
Chapitre 4 : Les Structures de Données	57
Introduction	57
Objectifs du cours.....	57
I. Les Listes	57
<i>I.1. Création d'une Liste.....</i>	<i>57</i>
<i>I.2. Manipulation des Listes</i>	<i>57</i>
<i>I.3. Exemple Pratique : Calcul de Moyenne à partir d'une Liste.....</i>	<i>58</i>
II. Les Tuples.....	58
<i>II.1. Création d'un Tuple</i>	<i>58</i>
<i>II.2. Utilisation des Tuples</i>	<i>58</i>
III. Les Dictionnaires	59
<i>III.1. Création d'un Dictionnaire</i>	<i>59</i>
<i>III.2. Manipulation des Dictionnaires</i>	<i>59</i>
<i>III.3. Exemple Pratique : Statistiques d'étudiants.....</i>	<i>59</i>
IV. Les Ensembles	59
<i>IV.1. Création et Manipulation d'un Ensemble</i>	<i>60</i>
<i>IV.2. Opérations sur les Ensembles</i>	<i>60</i>
V. Compréhensions de Listes et Dictionnaires.....	60
<i>V.1. Compréhension de Listes</i>	<i>60</i>
<i>V.2. Compréhension de Dictionnaires</i>	<i>61</i>

V.3.	<i>Exemple Pratique : Dictionnaire des Puissances d'un Nombre</i>	61
VI.	Exercices corrigés	61
VI.1.	<i>Exercice 1 : Manipulation de Listes</i>	61
VI.2.	<i>Exercice 2 : Statistiques avec Dictionnaire</i>	61
VI.3.	<i>Exercice 3 : Compréhensions</i>	61
VII.	Corrigé des Exercices	61
VII.1.	<i>Exercice 1 :</i>	61
VII.2.	<i>Exercice 2 :</i>	62
VII.3.	<i>Exercice 3 :</i>	62
VIII.	Conclusion	63
Chapitre 5 : Programmation Orientée Objet		63
I.	Introduction à la Programmation Orientée Objet	63
I.1.	Qu'est-ce que la Programmation Orientée Objet ?	63
I.2.	Concepts clés de la POO :	63
II.	Les Classes et Objets en Python	64
II.1.	Définir une Classe et Créer des Objets	64
II.2.	Héritage	64
II.3.	Encapsulation en Python	66
II.4.	Polymorphisme en Python	66
III.	Exemples Pratiques en Mathématiques	67
III.1.	Classe pour Modéliser un Vecteur	67
III.2.	Classe pour Modéliser une Matrice	69
III.3.	Système de gestion d'étudiants	71
Chapitre 6 : Interface Graphique avec Tkinter		72
Présentation de Tkinter		72
Qu'est-ce que Tkinter ?		72
Création de fenêtres simples		72

Ajout de Widgets.....	73
Les Labels	73
Les Boutons.....	73
Les zones de texte	74
Application : Interface de saisie et affichage de texte.....	74
Gestion des événements	75
Introduction aux événements	75
Gestion des clics de souris	75
Gestion de la saisie clavier	76
Application	77
Application 1 : Création d'une calculatrice simple	77
Application 2 : Création d'une Interface pour Saisir des Informations et les Afficher dans une Zone de Texte	79
Application 3 : dire bonjour dans une zone de texte	80
Application 4 : menu contextuel grâce à un clic droit.....	81

Chapitre 0 : Installation et utilisation de Python

Comment installer Python

Python est un langage de programmation populaire de haut niveau et à usage général. Python est un langage de programmation qui permet un développement rapide ainsi qu'une intégration système plus efficace. Python a deux versions principales différentes : Python 2 et Python 3. Les deux sont vraiment différents.

Python développe de nouvelles versions avec des modifications périodiques et les publie en fonction des numéros de version. Python en est actuellement à **la version 3.12.0**

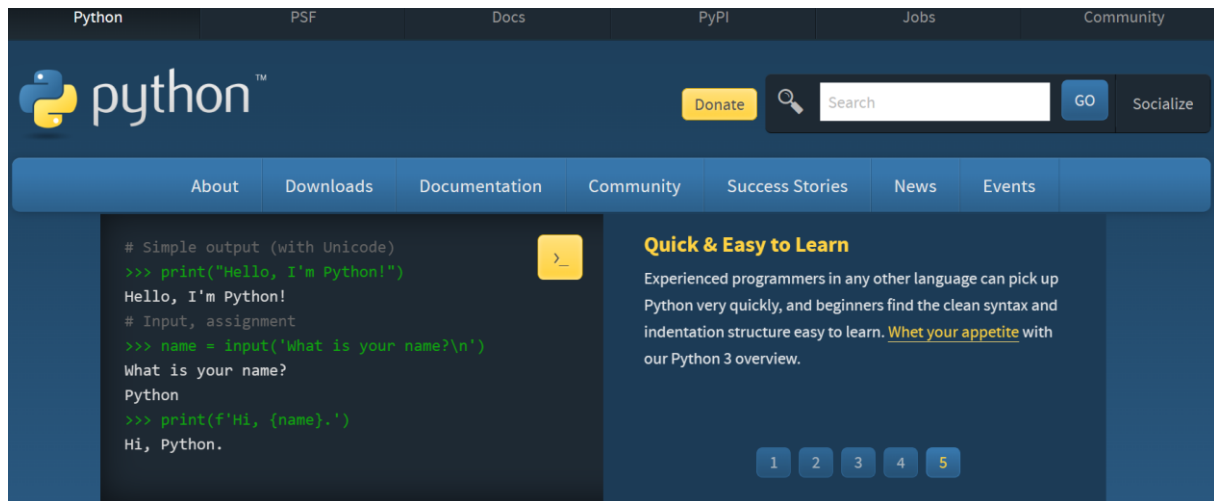
Python est beaucoup plus simple à apprendre et à programmer. N'importe quel éditeur de texte brut, tel que le bloc-notes ou le bloc-notes++, peut être utilisé pour créer des programmes Python. Pour faciliter la création de ces routines, on peut également utiliser un IDE en ligne pour Python ou même en installer un sur sa machine. Les IDE offrent une variété d'outils, y compris un éditeur de code convivial, le débogueur, le compilateur, etc.

Il faut avoir Python installé sur son système afin de commencer à créer du code Python et à effectuer de nombreuses procédures fascinantes et utiles. La première étape de l'apprentissage de la programmation en Python consiste à installer ou à mettre à jour Python sur votre ordinateur. Il existe plusieurs façons d'installer Python : vous pouvez utiliser un gestionnaire de paquets, obtenir des versions officielles de Python.org ou installer des versions spécialisées pour les appareils embarqués, le calcul scientifique et l'Internet des objets.

Pour devenir développeur Python, la première étape consiste à apprendre à installer ou à mettre à jour Python sur une machine ou un ordinateur local. Dans ce tutoriel, nous allons aborder l'installation de Python sur différents systèmes d'exploitation.

Installation sous Windows

Cliquez sur le lien <https://www.python.org> pour télécharger la dernière version de Python. Dans ce processus, nous allons installer Python 3.11.3 sur notre système d'exploitation Windows. Lorsque nous cliquons sur le lien ci-dessus, nous accédons à la page suivante.



Étape - 1 : Sélectionnez la version de Python à télécharger.

Cliquez sur le bouton de téléchargement [Download] pour télécharger le fichier exe de Python.



Si vous souhaitez télécharger la version spécifique de Python. Ensuite, vous pouvez faire défiler plus bas ci-dessous pour voir les différentes versions de 2 et 3 respectivement. Cliquez sur le bouton de téléchargement juste à côté du numéro de version que vous souhaitez télécharger.

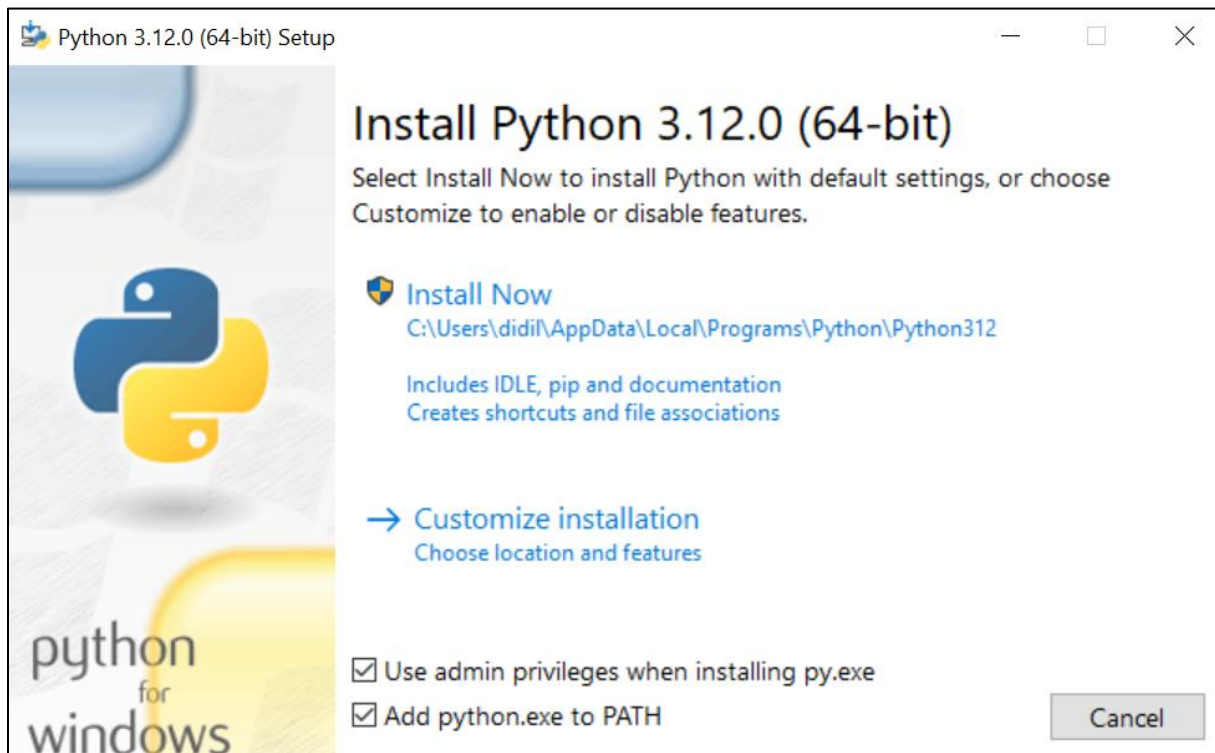
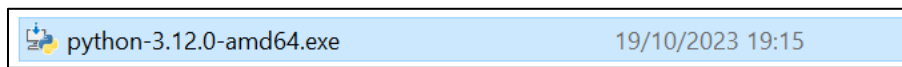
Looking for a specific release?

Python releases by version number:

Release version	Release date		Click for more
Python 3.12.0	Oct. 2, 2023	Download	Release Notes
Python 3.11.6	Oct. 2, 2023	Download	Release Notes
Python 3.11.5	Aug. 24, 2023	Download	Release Notes
Python 3.10.13	Aug. 24, 2023	Download	Release Notes
Python 3.9.18	Aug. 24, 2023	Download	Release Notes
Python 3.8.18	Aug. 24, 2023	Download	Release Notes
Python 3.10.12	June 6, 2023	Download	Release Notes

Étape - 2 : Cliquez sur le bouton Installer maintenant

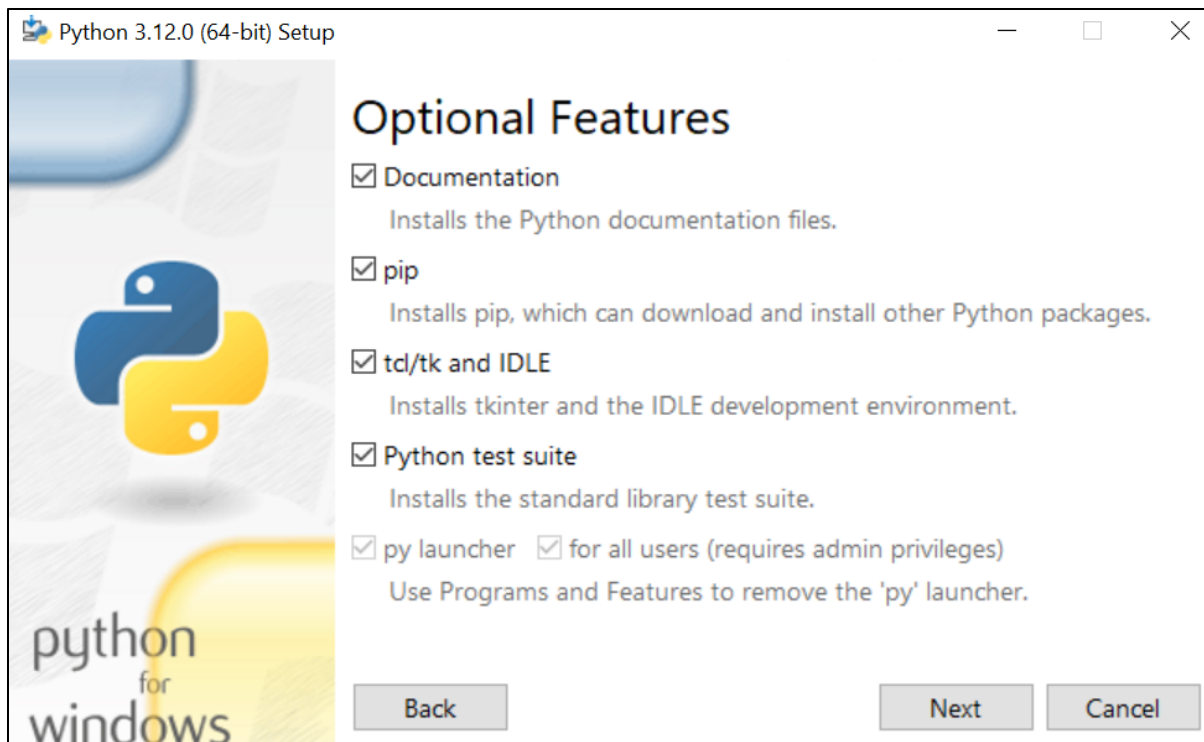
Double-cliquez sur le fichier exécutable téléchargé.



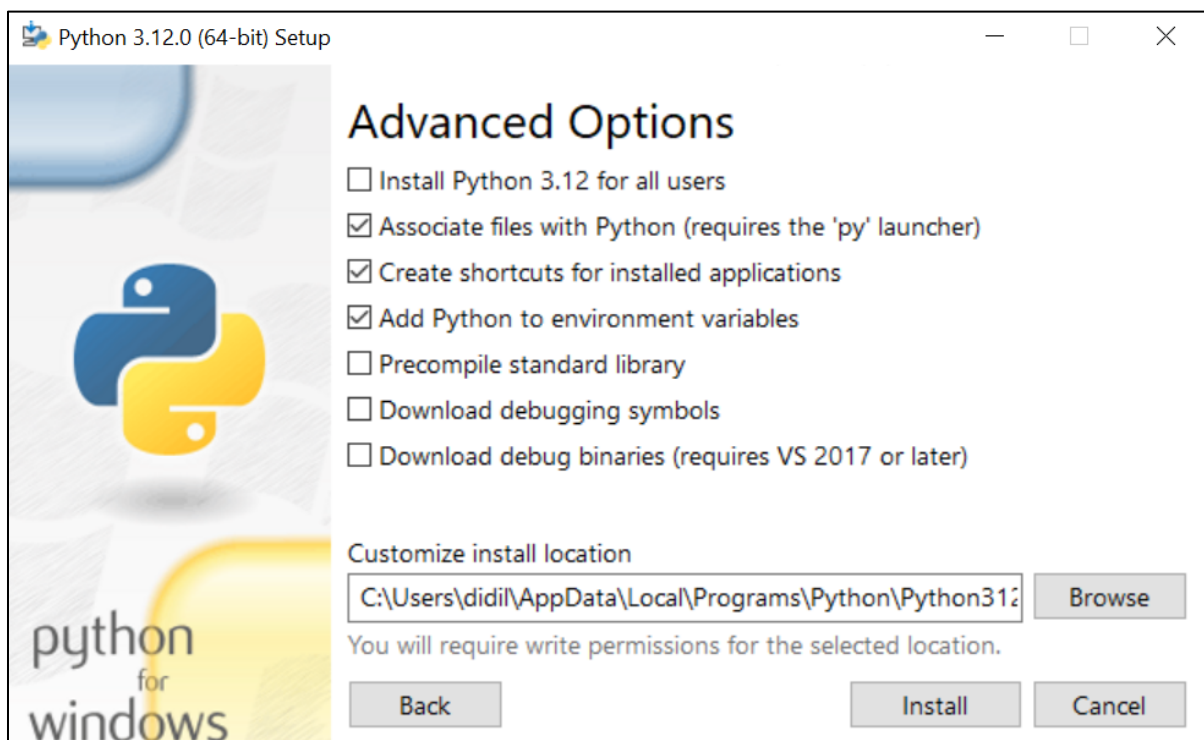
La fenêtre suivante s'ouvre. Cliquez sur la case à cocher Ajouter un chemin, cela définira automatiquement le chemin Python.

Maintenant, sélectionnez Personnaliser l'installation et continuez. Nous pouvons également cliquer sur personnaliser l'installation pour choisir l'emplacement et les fonctionnalités souhaités. Une autre chose importante est d'installer le lanceur pour que tous les utilisateurs soient cochés.

Ici, sous les options avancées, cliquez sur les cases à cocher de « Installer Python 3.12 pour tous les utilisateurs », qui n'était pas archivée auparavant. Cela vérifiera l'autre option « Précompiler la bibliothèque standard » automatiquement. Et l'emplacement de l'installation sera également modifié. Nous pouvons le modifier plus tard, nous laissons donc l'emplacement d'installation par défaut. Ensuite, cliquez sur le bouton d'installation pour enfin installer.

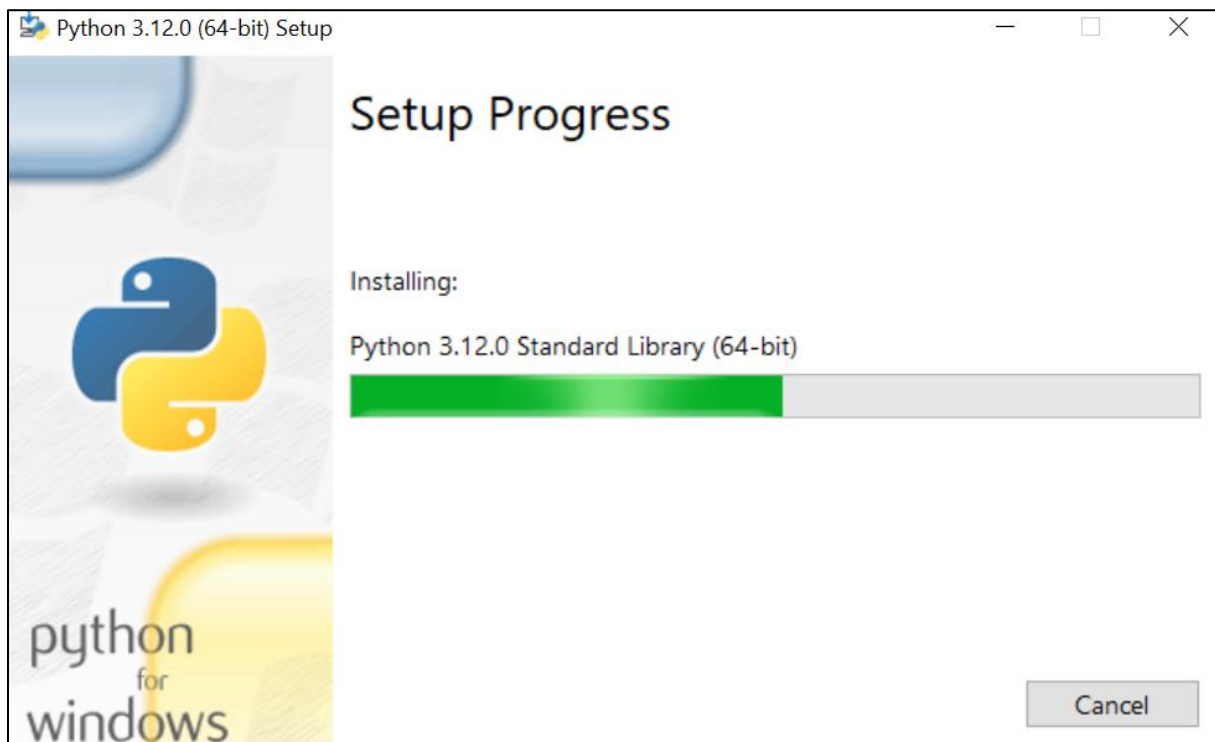


Laisser tous par défaut puis cliquer sur « **Next** »

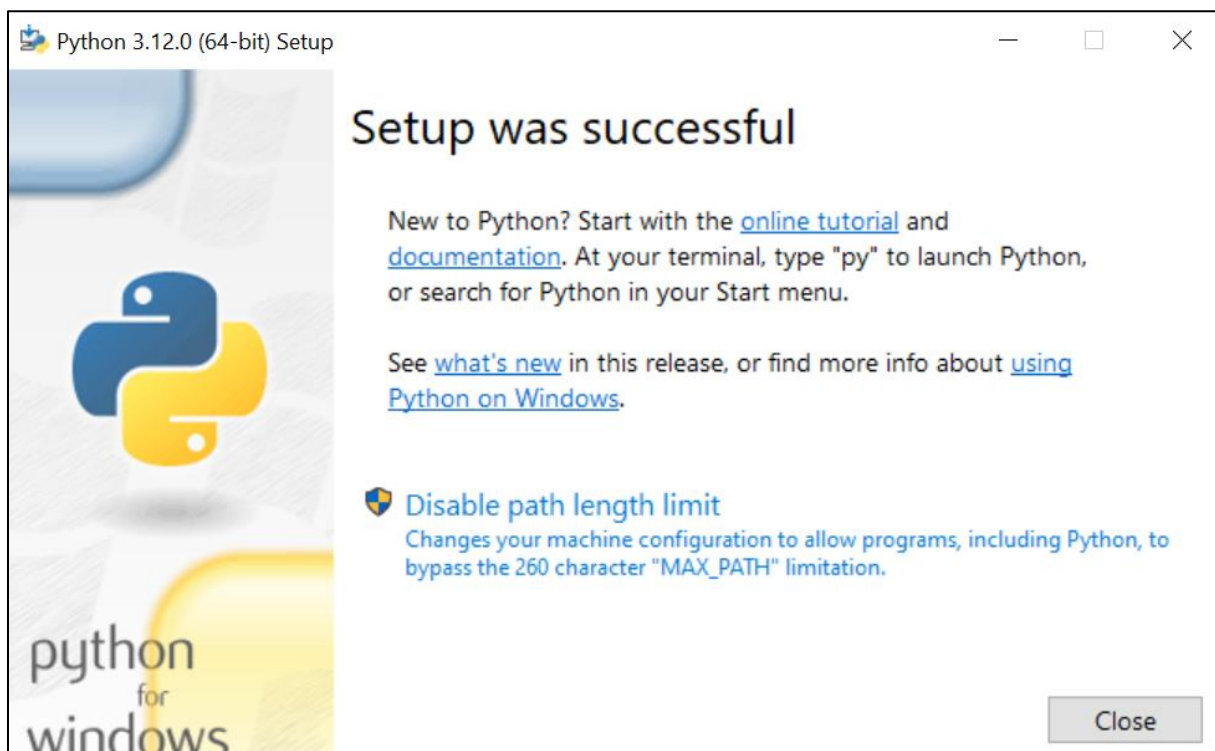


On clique sur le bouton « **install** » pour démarrer installation automatiquement (laisser par défaut)

Étape - 3 Installation en cours



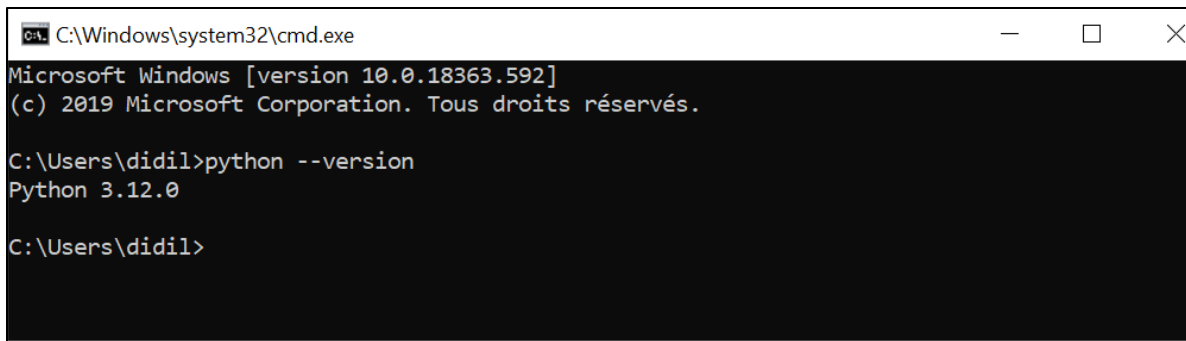
La mise en place est en cours. Toutes les bibliothèques python, les packages et les autres fichiers python par défaut seront installés dans notre système. Une fois l'installation réussie, la page suivante apparaîtra indiquant « L'installation a réussi ».



Étape - 4 : Vérification de l'installation de Python

Pour vérifier si le python est installé ou non dans notre système, nous devons procéder comme suit.

- Allez sur le bouton « **Démarrer** » et recherchez « **cmd** ».
- Tapez ensuite « **python - - version** ».
- Si python est installé avec succès, nous pouvons voir la version du python installée.
- S'il n'est pas installé, il affichera l'erreur comme « **'python' n'est pas reconnu comme une commande interne ou externe, un programme utilisable ou un fichier batch.** »



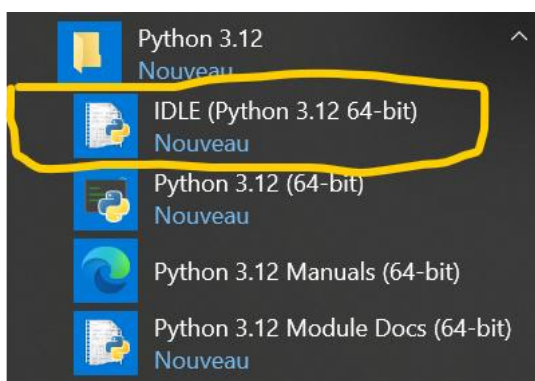
```
C:\Windows\system32\cmd.exe
Microsoft Windows [version 10.0.18363.592]
(c) 2019 Microsoft Corporation. Tous droits réservés.

C:\Users\didil>python --version
Python 3.12.0

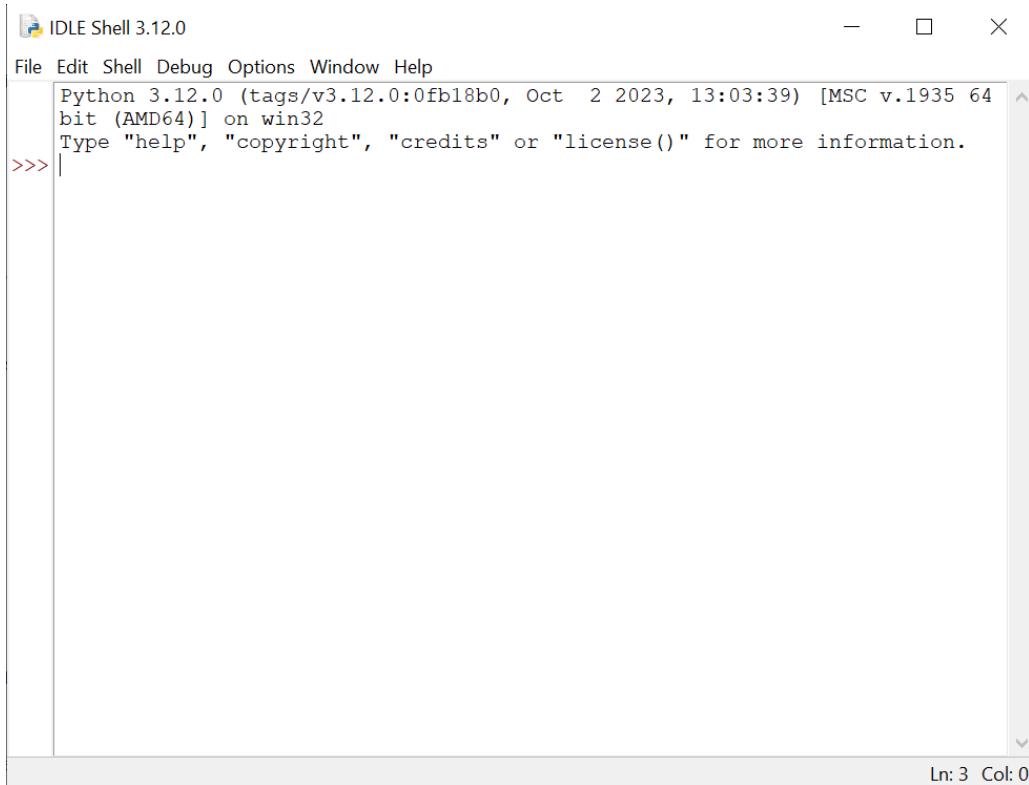
C:\Users\didil>
```

Étape - 5 : Ouverture de IDLE

Aller sur le menu démarrer  puis chercher le dossiers Python 3.12 et sélectionner IDLE (Python 3.12 64-bits) (Python 3.12 64-bits)



Maintenant, pour travailler sur notre premier programme python, nous allons aller à l'invite de l'interpréteur interactif (idle). Pour l'ouvrir, allez dans « Démarrer » et tapez idle. Ensuite, cliquez sur ouvrir pour commencer à travailler sur idle.



```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64  
bit (AMD64)] on win32  
Type "help", "copyright", "credits" or "license()" for more information.  
>>> |
```

Ln: 3 Col: 0

Premier programme Python

Python nous fournit les deux façons d'exécuter un programme :

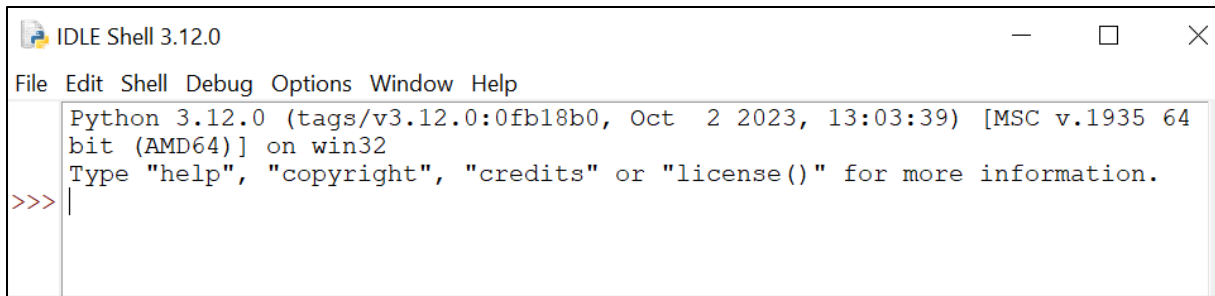
- Utilisation de l'invite de l'interpréteur interactif
- Utilisation d'un fichier de script

Invite interactive de l'interprète

Python nous fournit la fonctionnalité d'exécuter l'instruction Python une par une à l'invite interactive. C'est préférable dans le cas où nous sommes préoccupés par la sortie de chaque ligne de notre programme Python.

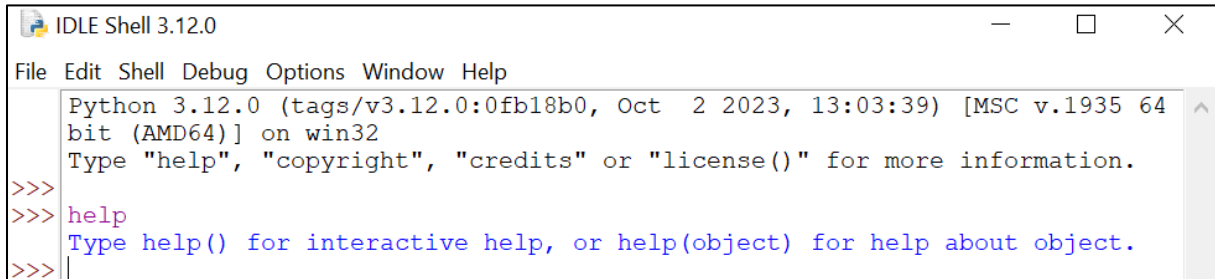
Pour ouvrir le mode interactif, ouvrez le terminal (ou l'invite de commande) et tapez python (python3 dans le cas où vous avez Python2 et Python3 tous deux installés sur votre système).

Cela ouvrira l'invite suivante dans laquelle nous pourrions exécuter l'instruction Python et vérifier leur impact sur la console.



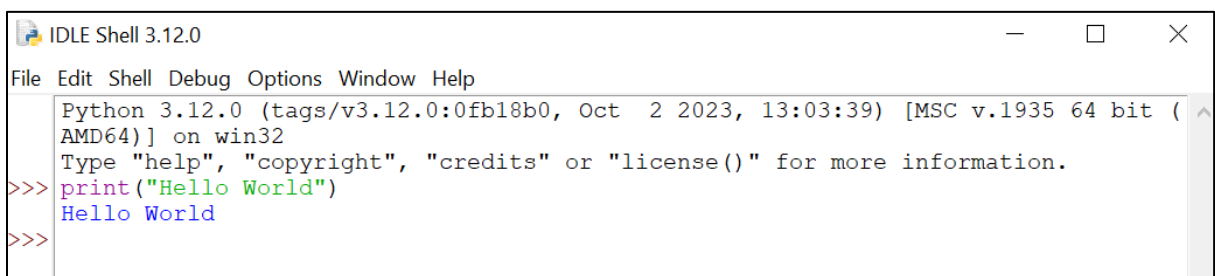
```
IDLE Shell 3.12.0
File Edit Shell Debug Options Window Help
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

Après avoir écrit l'instruction d'impression, appuyez sur la touche Entrée.



```
IDLE Shell 3.12.0
File Edit Shell Debug Options Window Help
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
>>> help
Type help() for interactive help, or help(object) for help about object.
>>> |
```

Tapons l'instructions suivantes :



```
IDLE Shell 3.12.0
File Edit Shell Debug Options Window Help
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>> |
```

Ici, nous obtenons le message « Hello World ! » imprimé sur la console.

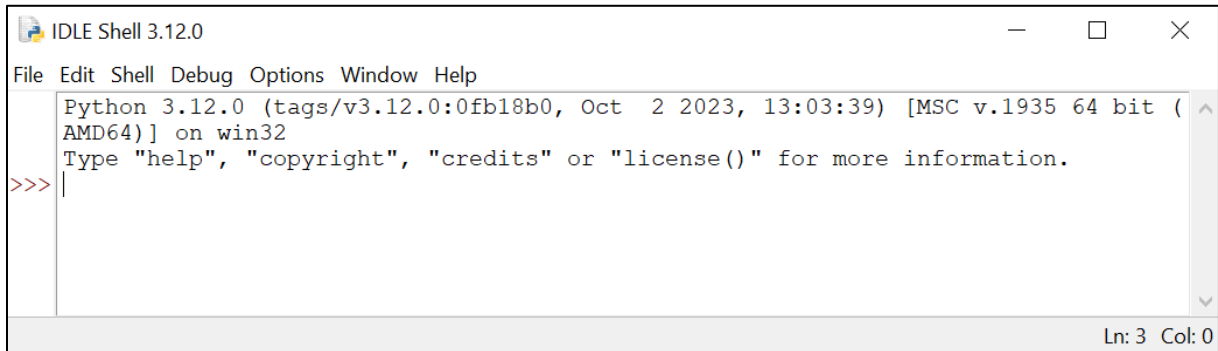
Utilisation d'un fichier de script (programmation en mode script)

Il est préférable d'utiliser l'invite de l'interpréteur pour exécuter les instructions d'une seule ligne du code. Cependant, nous ne pouvons pas écrire le code à chaque fois sur le terminal. Il n'est pas adapté à l'écriture de plusieurs lignes de code.

En utilisant le mode script, nous pouvons écrire plusieurs lignes de code dans un fichier qui peut être exécuté plus tard. Pour cela, nous devons ouvrir un éditeur comme le bloc-notes, créer un fichier nommé et l'enregistrer avec l'extension .py, qui signifie « Python ». Maintenant, nous allons implémenter l'exemple ci-dessus en utilisant le mode script.

`print (« Bonjour tout le monde »)` ; *#here, nous avons utilisé la fonction print() pour afficher le message sur la console.*

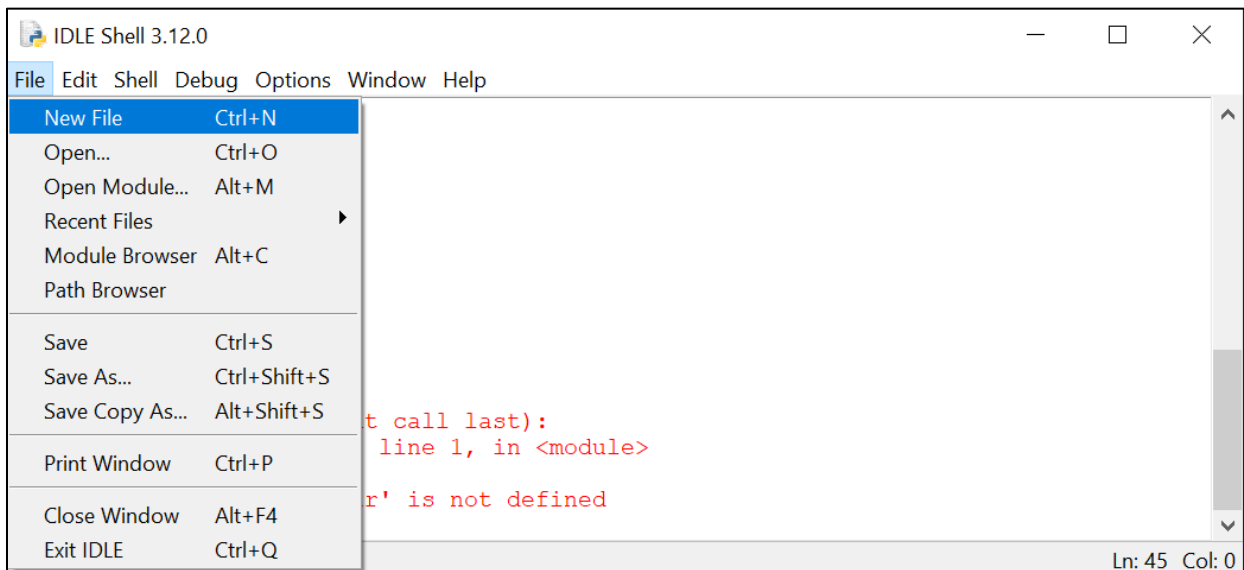
Pour exécuter ce fichier nommé first.py, nous devons exécuter la commande suivante sur le terminal.



```
Python 3.12.0 (tags/v3.12.0:0fb18b0, Oct 2 2023, 13:03:39) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> |
```

Ln: 3 Col: 0

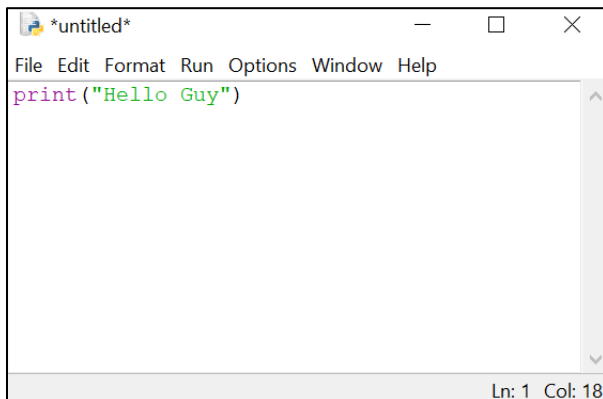
Étape - 1 : Ouvrez le shell interactif Python, et cliquez sur « **Fichier** » puis choisissez « **Nouveau** », cela ouvrira un nouveau script vierge dans lequel nous pourrons écrire notre code.



```
File Edit Shell Debug Options Window Help
New File Ctrl+N
Open... Ctrl+O
Open Module... Alt+M
Recent Files
Module Browser Alt+C
Path Browser
Save Ctrl+S
Save As... Ctrl+Shift+S
Save Copy As... Alt+Shift+S
Print Window Ctrl+P
Close Window Alt+F4
Exit IDLE Ctrl+Q
```

Ln: 45 Col: 0

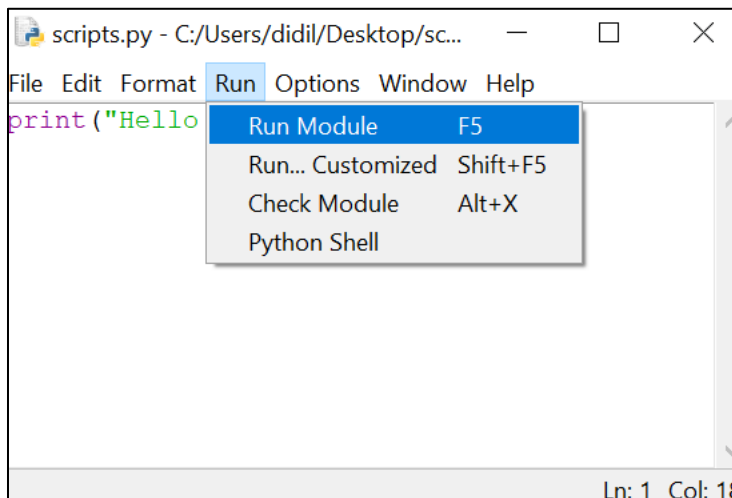
Étape -2 : Maintenant, écrivez le code et appuyez sur « **Ctrl + S** » pour enregistrer le fichier.



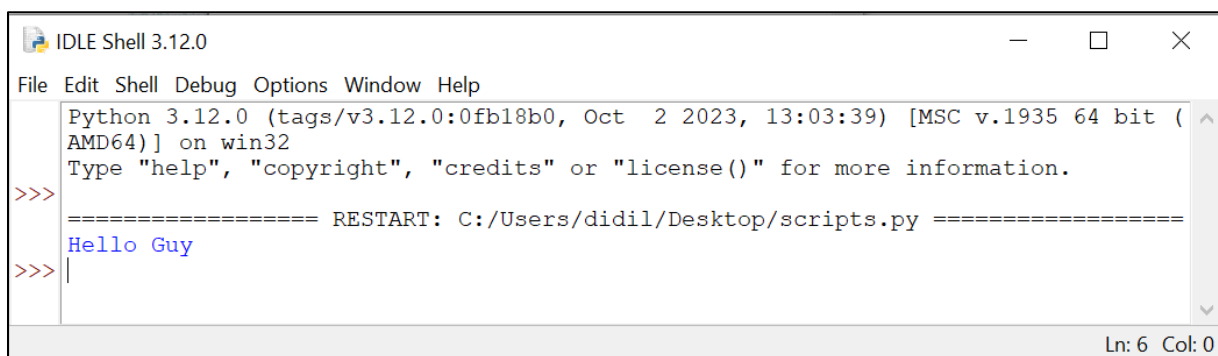
```
File Edit Format Run Options Window Help
print("Hello Guy")
```

Ln: 1 Col: 18

Étape - 3 : Après avoir enregistré le code, nous pouvons l'exécuter en cliquant sur « Exécuter » ou « **Exécuter le module** ». Il affichera la sortie vers le shell.



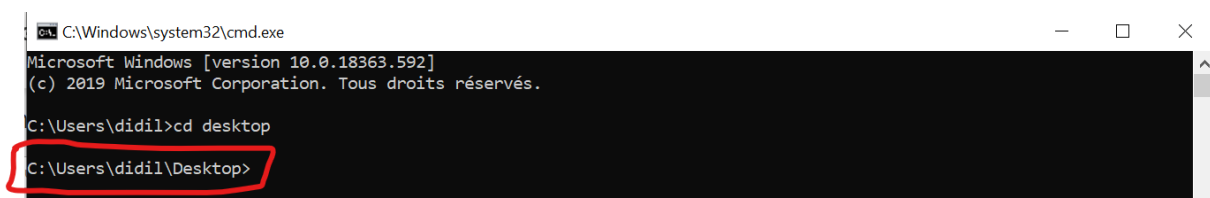
La sortie s'affichera comme suit.



Étape - 4 : En dehors de cela, nous pouvons également exécuter le fichier à l'aide du terminal du système d'exploitation. Mais, nous devons être conscients du chemin du répertoire où nous avons enregistré notre fichier.

Ouvrez l'invite de ligne de commande et accédez au répertoire, on suit les étapes suivantes :

➔ `cd desktop`



On est dans le repertoire Desktop : **C:\Users\didil\Desktop**

➔ `Dir`

```
C:\Users\didil\Desktop>Dir
Le volume dans le lecteur C n'a pas de nom.
Le numéro de série du volume est 3268-E82E

Répertoire de C:\Users\didil\Desktop

19/10/2023  20:54  <DIR>      .
19/10/2023  20:54  <DIR>      ..
10/10/2023  13:26  <DIR>      BASIC_VS
19/10/2023  16:36  <DIR>      COURS_IUA
16/10/2023  11:54  <DIR>      Ebook - Création de bases de données
09/10/2023  15:11  <DIR>      Internet Download Manager (IDM) v6.41 Build 20 + Fix [Lifetime Activation] {CracksHa
sh}
08/10/2023  15:10  <DIR>      RevoUninstaller_Portable
19/10/2023  20:54          20 scripts.py
17/10/2023  11:33      1 389 912 td1.pdf
08/10/2023  20:02  <DIR>      TEST1
18/10/2018  14:38  <DIR>      VB.NET-Mastering-Tutorial-Series-master
10/10/2023  13:44      13 863 716 VB.NET-Mastering-Tutorial-Series-master.zip
               3 fichier(s)          15 253 648 octets
               9 Rép(s)  945 443 393 536 octets libres
```

Elle permet de voir les dossiers et les fichiers qui sont le répertoire desktop, dans on va s'intéressé aux fichiers : **scripts.py**

Nous devons taper le mot-clé python, suivi du nom du fichier et appuyer sur Entrée pour exécuter le fichier Python.

➔ python scripts.py

```
C:\Windows\system32\cmd.exe

C:\Users\didil\Desktop>python scripts.py
Hello Guy

C:\Users\didil\Desktop>
```

Instructions multilignes

Les instructions multilignes sont écrites dans le bloc-notes comme un éditeur et enregistrées avec l'extension .py. Dans l'exemple suivant, nous avons défini l'exécution des plusieurs lignes de code à l'aide du script Python.

```
name = "Andrew Venis"

branch = "Computer Science"

age = "25"

print("My name is: ", name, )

print("My age is: ", age)
```

```
C:\Users\didil\Desktop>python scripts.py
My name is: Andrew Venis
My age is: 25
```

Avantages et inconvénients du mode script

Le mode script présente également peu d'avantages et d'inconvénients. Comprenons les avantages suivants de l'exécution de code en mode script.

- Nous pouvons exécuter plusieurs lignes de code.
- Le débogage est facile en mode script.
- Il convient aussi bien aux débutants qu'aux experts.
- Voyons les inconvénients du mode script.

Nous devons enregistrer le code à chaque fois si nous apportons une modification au code. Cela peut être fastidieux lorsque nous exécutons une seule ou quelques lignes de code.

Premiers pas avec PyCharm

Sur Windows, nous avons une alternative comme bloc-notes ou bloc-notes++ pour modifier le code. Cependant, ces éditeurs ne sont pas utilisés comme IDE pour python car ils sont incapables d'afficher les suggestions liées à la syntaxe.

JetBrains fournit l'IDE multiplateforme le plus populaire et le plus largement utilisé, PyCharm, pour exécuter les programmes python.

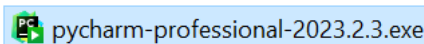
Installation de PyCharm

Comme nous l'avons déjà dit, PyCharm est un IDE multiplateforme, et peut donc être installé sur une variété de systèmes d'exploitation. Dans cette section du didacticiel, nous couvrirons le processus d'installation de PyCharm sur Windows.

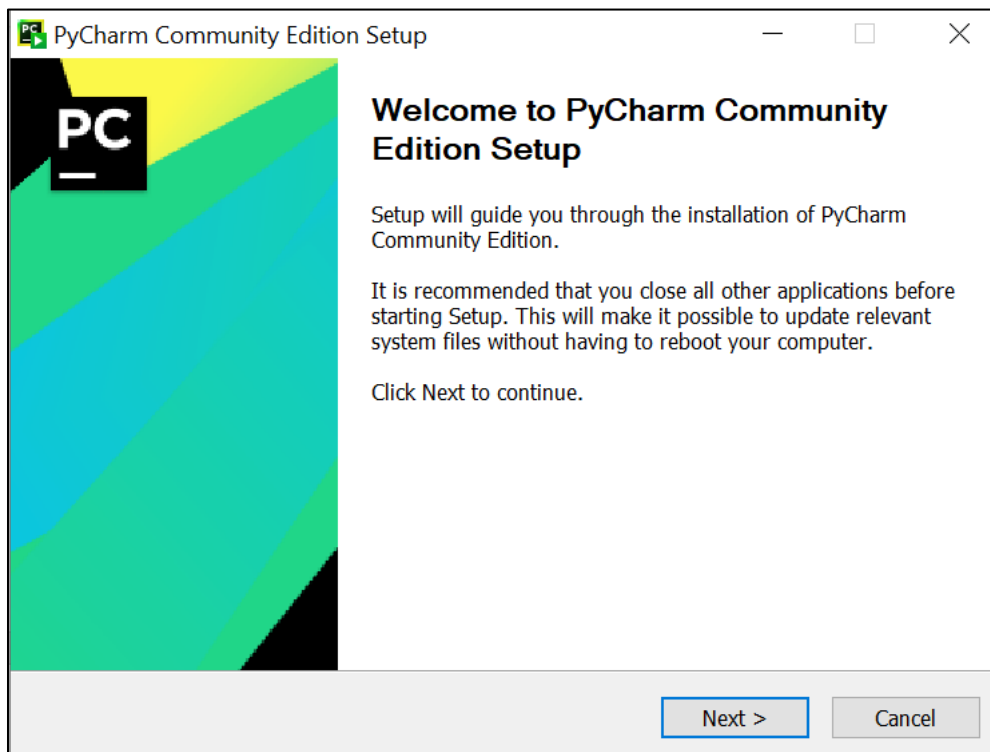
L'installation de PyCharm sur Windows est très simple. Pour installer PyCharm sur le système d'exploitation Windows, visitez le lien :

<https://www.jetbrains.com/pycharm/download/download-thanks.html?platform=windows>

pour télécharger le programme d'installation exécutable. Double-cliquez sur le fichier d'installation (.exe) et installez PyCharm en cliquant sur suivant à chaque étape.

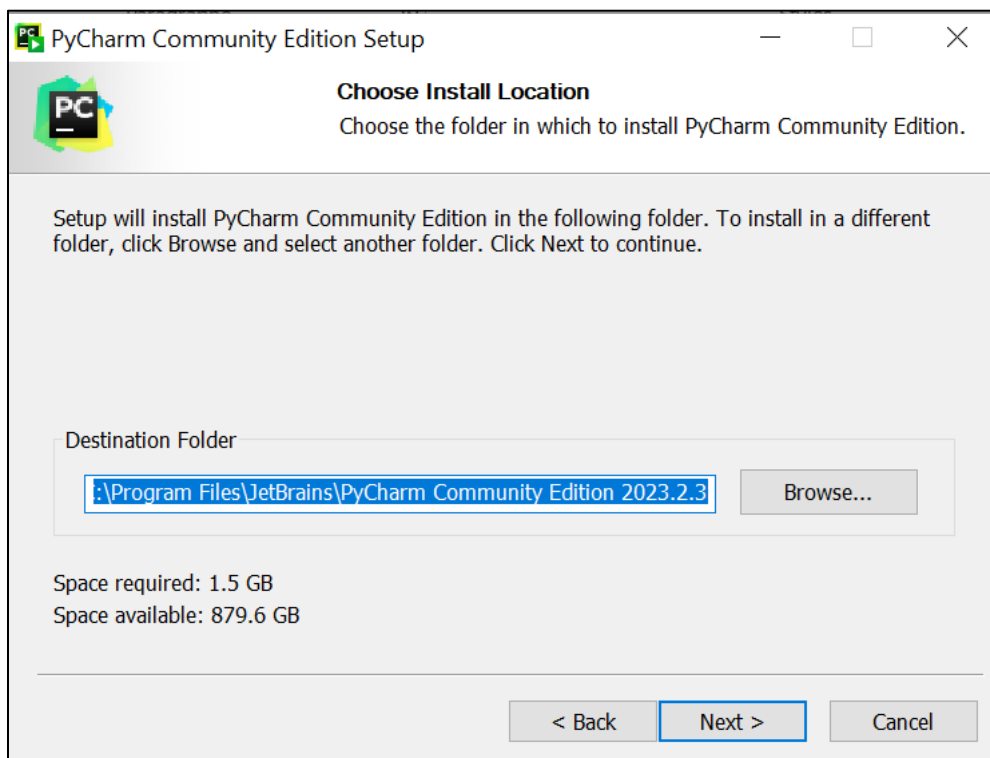


Etape 1 : Bienvenue



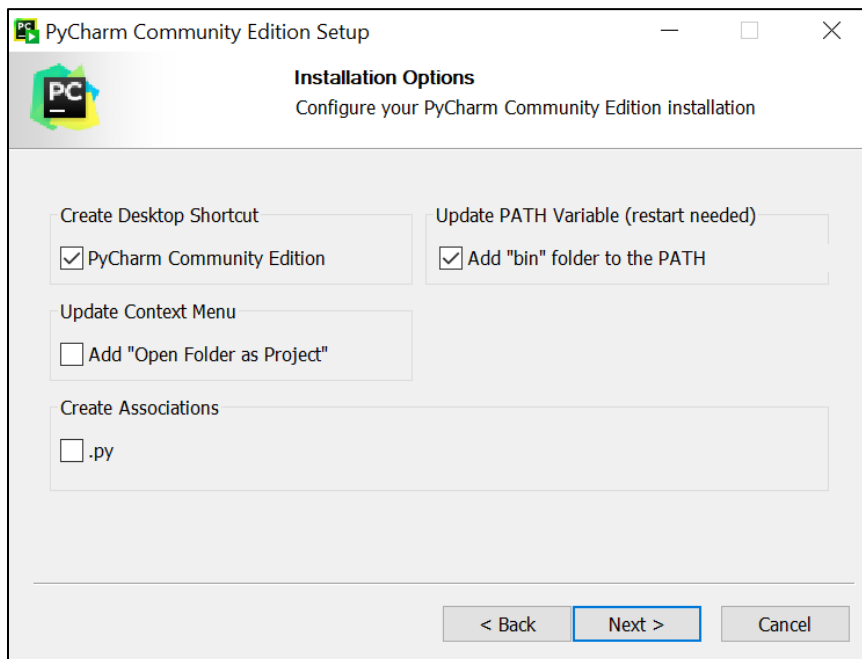
Cliquez sur « Next »

Etape 2 : choisir le fichier d'installation



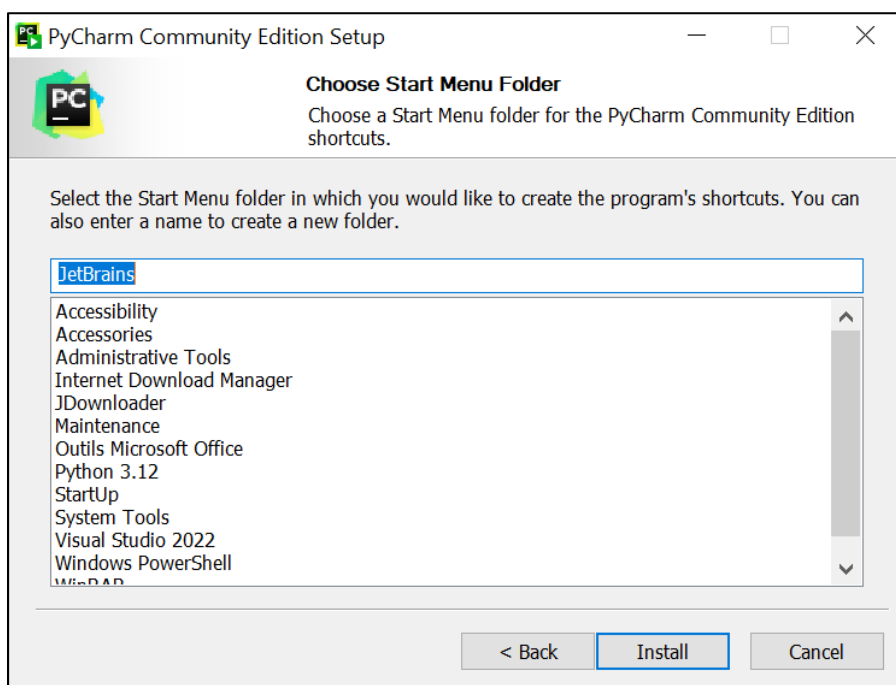
Laissez le chemin par défaut, puis cliquer sur « Next »

Etape 3 : option d'installation



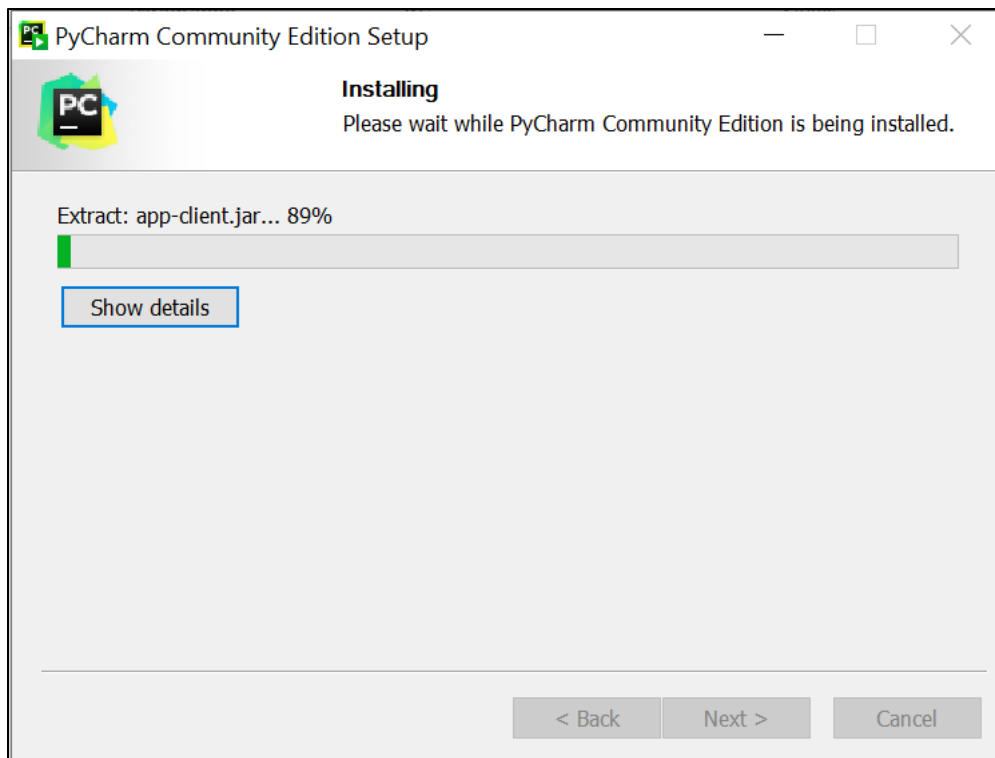
Cocher les deux cases ci-dessus puis cliquez sur « Next »

Etape 4 : Choisir le Menu du dossier

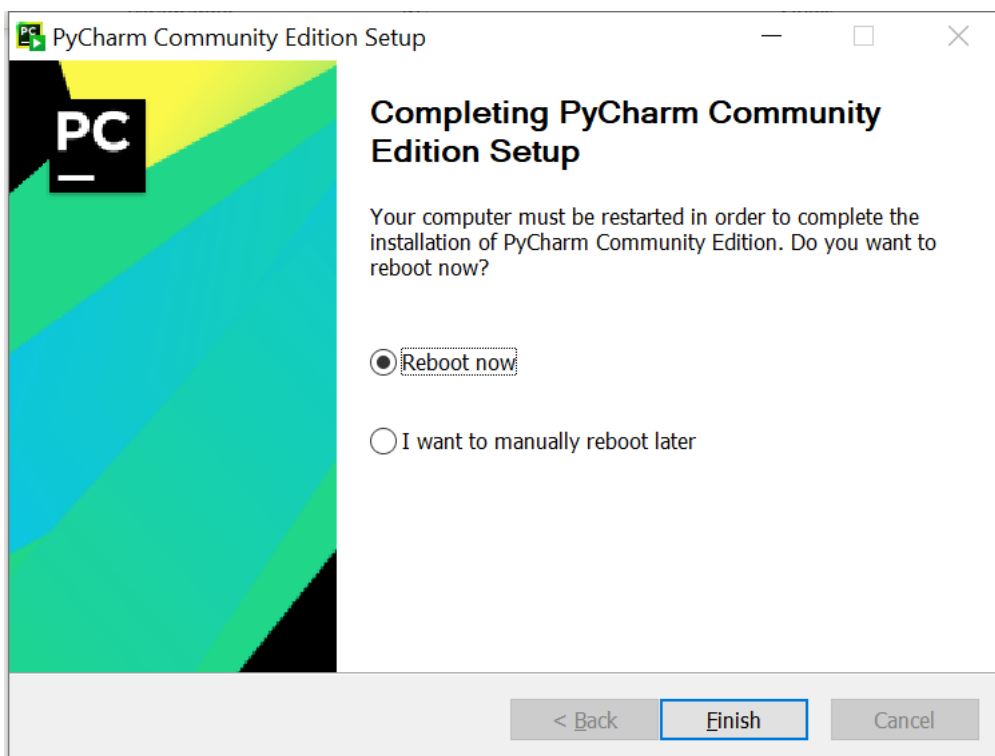


Laisser le paramètre par défaut « JetBrains », puis cliquez sur « Install »

Etape 5 : installation de Pycharm



Etape 6 : Installation Complète

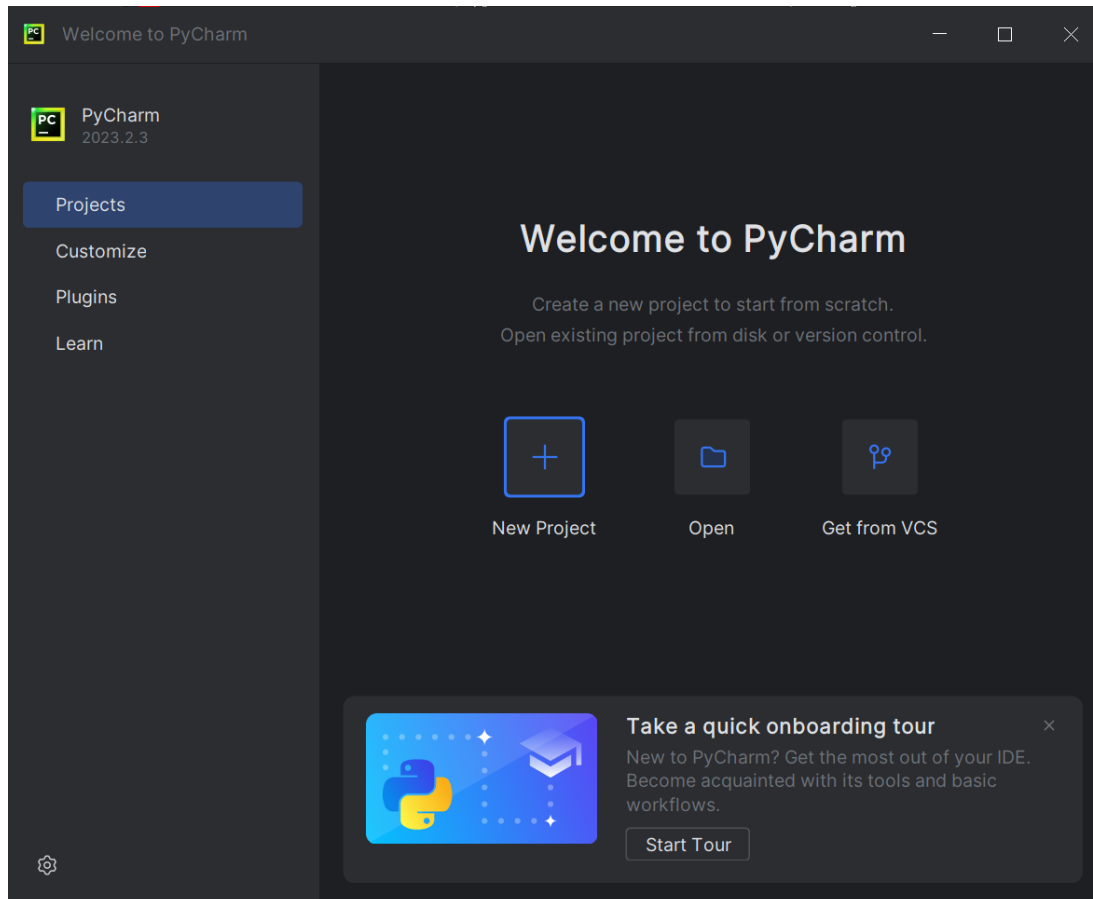


Cocher la case « **Reboot now** »

Premiers programmes sur Pycharm

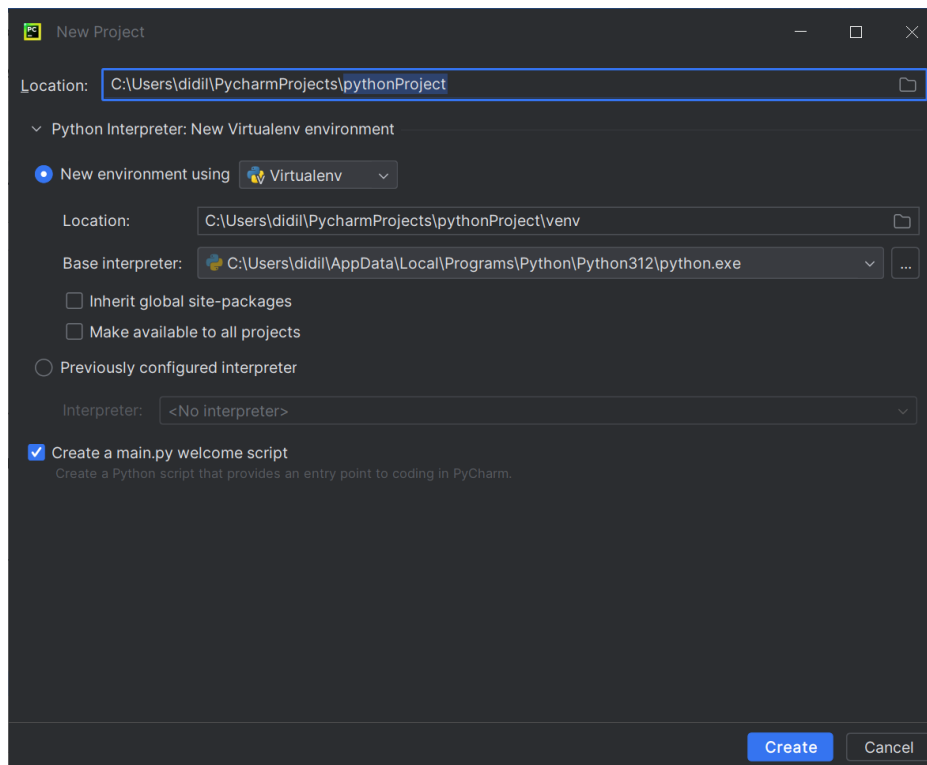
Pour créer un premier programme sur Pycharm, suivez l'étape suivante.

Étape - 1. Ouvrez l'éditeur Pycharm. Cliquez sur l'option « Créer un nouveau projet » pour créer un nouveau projet.

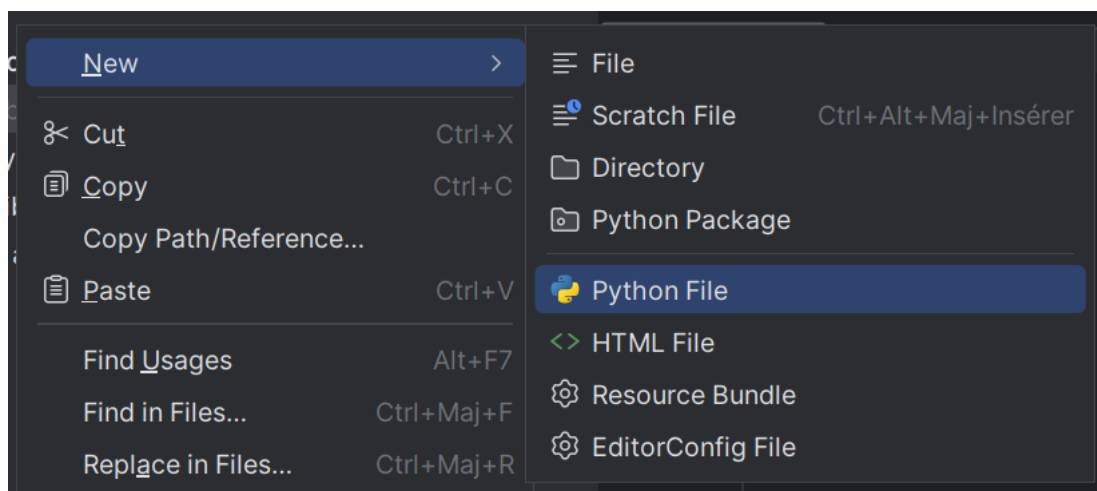


Étape - 2. Sélectionnez un emplacement pour enregistrer le projet.

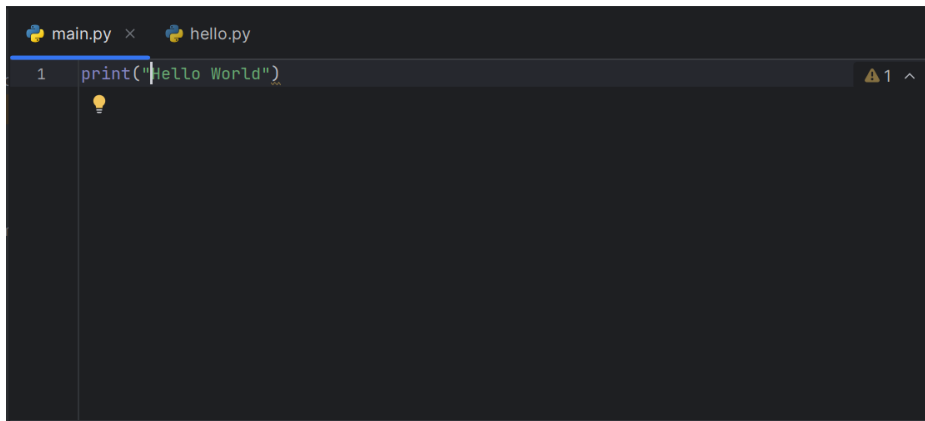
- Nous pouvons enregistrer le projet nouvellement créé à l'emplacement de mémoire souhaité ou conserver l'emplacement du fichier tel quel, mais au moins changer le nom par défaut du projet sans titre en « FirstProject » ou quelque chose de significatif.
- Pycharm a automatiquement trouvé l'interpréteur Python installé.
- Après avoir changé le nom, cliquez sur le bouton « Créer ».



Étape - 3. Cliquez sur le menu « Fichier » et sélectionnez « Nouveau ». En cliquant sur l'option « Nouveau », vous verrez différents formats de fichiers. Sélectionnez le « **Fichier Python** »

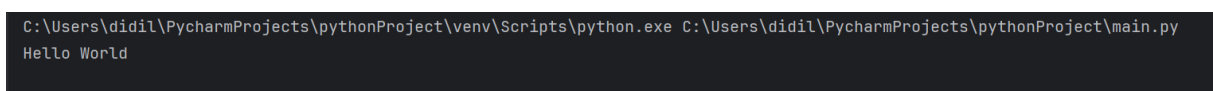


Étape - 5. Tapez maintenant le premier programme - print(« Hello World ») puis cliquez sur le menu « Exécuter » pour exécuter le programme.



```
main.py x hello.py
1 print("Hello World")
```

Étape - 6. La sortie apparaîtra en bas de l'écran.



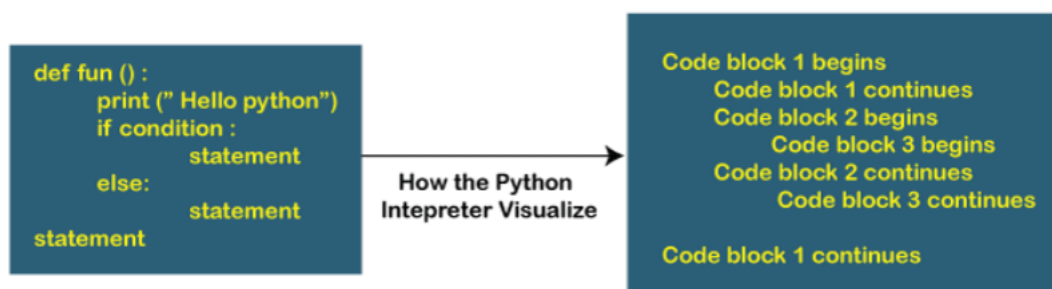
```
C:\Users\didil\PycharmProjects\pythonProject\venv\Scripts\python.exe C:\Users\didil\PycharmProjects\pythonProject\main.py
Hello World
```

Syntaxe de base de Python

Indentation et commentaire en Python

L'indentation est le concept le plus important du langage de programmation Python. Une mauvaise utilisation de l'indentation entraînera la création de « IndentationError » dans notre code.

L'indentation n'est rien d'autre que l'ajout d'espaces avant l'instruction lorsque cela est nécessaire. Sans indentation, Python ne sait pas vers quelle instruction être exécutée ensuite. L'indentation définit également quelles instructions appartiennent à quel bloc. S'il n'y a pas d'indentation ou une indentation incorrecte, il affichera « IndentationError » et interrompra notre code.



L'indentation Python définit le groupe particulier d'instructions appartenant au bloc particulier. Les langages de programmation tels que C, C++, java utilisent les accolades {} pour définir les blocs de code.

En Python, les instructions qui sont du même niveau à droite appartiennent au même bloc. Nous pouvons utiliser quatre espaces pour définir l'indentation. Voyons les lignes de code suivantes.

```
list1 = [1, 2, 3, 4, 5]

for i in list1:

    print(i)

    if i==4:

        break

print("End of for loop")
```

Sortie:

```
C:\Users\didil\PycharmProjects\pythonProject\venv\Scripts\python.exe C:\Users\didil\PycharmProjects\pythonProject\hello.py
1
2
3
4
End of for loop
```

Explication :

Dans le code ci-dessus, la boucle for a un bloc de code et if l'instruction a son bloc de code à l'intérieur de la boucle for. Les deux sont en retrait avec quatre espaces. La dernière instruction print() n'est pas indentée ; Cela signifie qu'il n'appartient pas à la boucle for.

Commentaires en Python

Les commentaires sont essentiels pour définir le code et nous aideront, ainsi que d'autres personnes, à comprendre le code. En regardant le commentaire, nous pouvons facilement comprendre l'intention de chaque ligne que nous avons écrite dans le code. Nous pouvons également trouver l'erreur très facilement, la corriger et l'utiliser dans d'autres applications.

En Python, nous pouvons appliquer des commentaires en utilisant le caractère de hachage #. L'interpréteur Python ignore complètement les lignes suivies d'un caractère dièse. Un bon programmeur utilise toujours les commentaires pour faire du code sous stable. Voyons l'exemple suivant d'un commentaire.

```
name = "Thomas" # Assignment d'une valeur de chaîne à la variable name
```

Nous pouvons ajouter un commentaire dans chaque ligne du code Python.

```
Frais = 10000 # les frais de cours sont 10000
```

```
Frais = 20000 # les frais de cours sont 20000
```

C'est une bonne idée d'ajouter du code dans n'importe quelle ligne de la section de code du code dont le but n'est pas évident. Il s'agit d'une bonne pratique à apprendre lors du codage.

Types de commentaires

Python offre la possibilité d'écrire des commentaires de deux manières : un commentaire sur une seule ligne et un commentaire sur plusieurs lignes.

Commentaire d'une seule ligne - Le commentaire d'une seule ligne commence par le caractère dièse # suivi d'un texte pour plus d'explications.

```
# définir les notes d'un élève
```

```
Repères = 90
```

Nous pouvons également écrire un commentaire à côté d'une instruction de code. Prenons l'exemple suivant.

```
Nom = "James" # le nom d'un élève est James
```

```
Note = 90 # définir la note de l'élève
```

```
Filière = "Computer Science" # définir la filière de l'élève
```

Commentaires multilignes - Python n'a pas de support explicite pour les commentaires multilignes, mais nous pouvons utiliser le caractère dièse # pour les lignes multiples.

```
# nous définissons la boucle for
```

```
# Pour itérer la liste donnée.
```

```
# Exécutez ce code.
```

Nous pouvons également utiliser un autre moyen.

```
"""
```

```
Il s'agit d'un exemple
```

```
De commentaire multiligne
```

Utilisation des guillemets triples

"""

Identificateurs Python

Les identificateurs Python font référence à un nom utilisé pour identifier une variable, une fonction, un module, une classe, un module ou d'autres objets. Il y a peu de règles à suivre lors de la dénomination de la variable Python.

- Un nom de variable doit commencer par une lettre anglaise ou un trait de soulignement (`_`).
- Un nom de variable ne peut pas commencer par le nombre.
- Les caractères spéciaux ne sont pas autorisés dans le nom de la variable.
- Le nom de la variable est sensible à la casse.

Comprenons l'exemple suivant.

```
number = 10  
  
print(num)  
  
_a = 100  
  
print(_a)  
  
x_y = 1000  
  
print(x_y)
```

Sortie :

```
C:\Users\didil\PycharmProjects\pythonProject\venv\Scripts\python.exe C:\Users\didil\PycharmProjects\pythonProject\hello.py  
10  
100  
1000
```

Nous avons défini la syntaxe de base du langage de programmation Python. Nous devons être familiers avec le concept de base de tous les langages de programmation. Une fois que nous avons mémorisé les concepts mentionnés ci-dessus. Le parcours d'apprentissage de Python deviendra plus facile.

Chapitre 1 : Notion Fondamentale en Python

Introduction

Python est un langage de programmation polyvalent et très utilisé dans les domaines scientifique et mathématique. Ce cours se concentre sur les bases du langage pour préparer les étudiants à des applications plus complexes.

Variables, Constantes et Littéraux

Variables

Les variables sont des conteneurs pour stocker des valeurs. En Python, vous pouvez créer une variable en lui attribuant une valeur.

```
# Déclaration d'une variable
```

```
nombre = 10
```

```
nom = "Alice"
```

```
pi = 3.14
```

Constantes

En Python, il n'y a pas de véritable concept de constantes comme dans d'autres langages (e.g., const en C++). Cependant, il est d'usage de nommer les variables constantes en utilisant des lettres majuscules.

```
PI = 3.14159
```

```
GRAVITE = 9.81
```

Littéraux

Les littéraux sont les valeurs directes dans le code. Ils peuvent être de différents types :

- **Entiers** : 5, 100
- **Flottants** : 3.14, 2.718
- **Chaînes de caractères** : "Bonjour", 'Python'

Exemple complet :

Littéraux

```
entier = 5

flottant = 3.14

chaine = "Python est puissant"

print(entier)

print(flottant)

print(chaine)
```

Exercice :

1. Créez une variable pour chaque type de littéral (entier, flottant, chaîne) et affichez-les.
2. Déclarez une "constante" pour le nombre d'or et affichez-la.

Correction :

```
# Variables pour chaque type de littéral

entier = 42

flottant = 2.718

chaine = "Mathématiques"

print(entier)    # 42

print(flottant)  # 2.718

print(chaine)    # Mathématiques

# Constante

NOMBRE_DOR = 1.618

print(NOMBRE_DOR) # 1.618
```

Types de Conversion

Python permet de convertir entre différents types de données. Voici quelques conversions courantes :

2.1 Conversion entre types de données :

```
# Convertir un entier en flottant

nombre_entier = 10

nombre_flottant = float(nombre_entier)

print(nombre_flottant) # 10.0

# Convertir un flottant en entier

nombre_flottant = 3.99

nombre_entier = int(nombre_flottant)

print(nombre_entier) # 3

# Convertir un entier en chaîne de caractères

nombre_entier = 42

chaine = str(nombre_entier)

print(chaine) # "42"
```

Exercice :

1. Convertissez un nombre flottant en entier et affichez le résultat.
2. Convertissez une chaîne de caractères en entier, puis ajoutez 10 et affichez le résultat.

Correction :

```
# Conversion flottant en entier

flottant = 7.89

entier = int(flottant)

print(entier) # 7

# Conversion chaîne en entier

chaine = "25"

entier = int(chaine) + 10

print(entier) # 35
```

3. Entrée et Sortie

3.1 Lecture de l'entrée utilisateur :

```
# Lire une entrée utilisateur  
  
nom = input("Entrez votre nom : ")  
  
print("Bonjour, " + nom)
```

3.2 Écriture à l'écran :

```
# Afficher un message  
  
age = 21  
  
print("Vous avez", age, "ans")
```

Exercice :

1. Écrivez un programme qui demande à l'utilisateur de saisir deux nombres, les additionne et affiche le résultat.
2. Demandez à l'utilisateur son prénom et son âge, puis affichez un message personnalisé.

Correction :

```
# Saisie de deux nombres et addition  
  
nombre1 = float(input("Entrez le premier nombre : "))  
  
nombre2 = float(input("Entrez le second nombre : "))  
  
somme = nombre1 + nombre2  
  
print("La somme est :", somme)  
  
# Saisie du prénom et de l'âge  
  
prenom = input("Quel est votre prénom ? ")  
  
age = int(input("Quel est votre âge ? "))  
  
print("Bonjour", prenom + ", vous avez", age, "ans.")
```

4. Opérateurs

4.1 Opérateurs arithmétiques :

```
a = 10

b = 5

# Addition

print(a + b) # 15

# Soustraction

print(a - b) # 5

# Multiplication

print(a * b) # 50

# Division

print(a / b) # 2.0

# Modulo

print(a % b) # 0

# Exponentiation

print(a ** b) # 100000
```

4.2 Opérateurs de comparaison :

```
x = 10

y = 20

print(x == y) # False

print(x != y) # True

print(x > y) # False

print(x < y) # True

print(x >= y) # False

print(x <= y) # True
```

Exercice :

1. Créez un programme qui demande à l'utilisateur deux nombres et affiche les résultats des opérations de base (addition, soustraction, multiplication, division).
2. Écrivez un programme qui compare deux nombres saisis par l'utilisateur et affiche le plus grand et le plus petit.

Correction :

```
# Opérations de base

nombre1 = float(input("Entrez le premier nombre : "))
nombre2 = float(input("Entrez le second nombre : "))

print("Addition :", nombre1 + nombre2)
print("Soustraction :", nombre1 - nombre2)
print("Multiplication :", nombre1 * nombre2)
print("Division :", nombre1 / nombre2)

# Comparaison de deux nombres

nombre1 = float(input("Entrez un premier nombre : "))
nombre2 = float(input("Entrez un second nombre : "))

if nombre1 > nombre2:
    print("Le plus grand nombre est :", nombre1)
    print("Le plus petit nombre est :", nombre2)
else:
    print("Le plus grand nombre est :", nombre2)
    print("Le plus petit nombre est :", nombre1)
```

Chapitre 2 : Les structures de contrôle

Nous allons couvrir les concepts suivants :

1. **Les conditions** : if, elif, else

2. **Les boucles** : for, while

3. **Exemples pratiques** : calcul de somme, tables de multiplication, évaluation de séries mathématiques

I. Introduction à Python pour les structures de contrôle

Python est un langage de programmation très puissant qui permet de résoudre des problèmes complexes en utilisant des concepts simples comme les **conditions** et les **boucles**. Ces structures de contrôle permettent de gérer le flux d'un programme.

II. Les Conditions : if, elif, else

Les conditions permettent de prendre des décisions dans un programme. Le principe est simple : **si une condition est vraie**, alors une action spécifique est exécutée ; sinon, d'autres actions peuvent être exécutées.

II.1. La condition if ... else

Une instruction if peut comporter une clause else facultative. L'instruction else est exécutée si la condition de l'instruction if est évaluée à faux.

Syntaxe :

```
if condition:
    # body of if statement
else:
    # body of else statement
```

Exemple 1 : Vérification d'un nombre positif

```
nombre = int(input('Enter a number: '))
if nombre > 0:
    print(f'{nombre} est positif')
else:
    print(f'{nombre} n'est pas positif')
print('Merci d\'avoir verifiez votre nombre')
```

Explication :

Ce programme vérifie si un nombre est pair. Si l'utilisateur saisit 10, la condition `nombre > 0` s'évalue à Vrai. Par conséquent, le corps de `if` est exécuté et le corps de `else` est ignoré. Par contre, si l'utilisateur saisit 0, la condition `nombre > 0` est évaluée à False. Par conséquent, le corps de `if` est ignoré et le corps de `else` est exécuté.

Exemple 2 : Vérification d'un nombre pair ou impair

```
nombre = int(input("Entrez un nombre : "))
if nombre % 2 == 0:
    print(f"{nombre} est un nombre pair.")
else:
    print(f"{nombre} est un nombre impair.")
```

Explication :

Ce programme vérifie si un nombre est divisible par 2 en utilisant l'opérateur modulo `%`. Si le reste est 0, le nombre est pair, sinon il est impair.

II.2. Condition if...elif...else

Syntaxe :

```
if condition:
    # instructions exécutées si la condition est vraie
elif autre_condition:
    # instructions exécutées si la deuxième condition est vraie
else:
    # instructions exécutées si aucune condition précédente n'est vraie
```

Exemples concrets

Exemple 1 : Classification d'une note

```
note = float(input("Entrez la note : "))
if note >= 16:
```

```
print("Très bien")  
  
elif note >= 14:  
  
    print("Bien")  
  
elif note >= 12:  
  
    print("Assez bien")  
  
elif note >= 10:  
  
    print("Passable")  
  
else:  
  
    print("Échec")
```

Explication : Ce programme utilise plusieurs conditions (elif) pour classer une note selon sa valeur.

Exemple 2 : Vérifions si un nombre est positif, négatif, ou nul

```
nombre = int(input("Entrez un nombre : "))  
  
if nombre > 0:  
  
    print(f"{nombre} est un nombre positif.")  
  
elif nombre < 0:  
  
    print(f"{nombre} est un nombre négatif.")  
  
else:  
  
    print(f"{nombre} est nulle.")  
  
print('merci pour ce test')
```

Explication :

Ici, la première condition, `nombre > 0`, est évaluée à `False`. Dans ce scénario, la deuxième condition est vérifiée : la deuxième condition, `nombre < 0`, est évaluée à `Vraie`. Par conséquent, les instructions à l'intérieur du bloc `elif` sont exécutées. Dans le programme ci-dessus, il est important de noter que, quelle que soit la valeur de la variable `nombre`, un seul bloc de code sera exécuté.

III. Les boucles : for et while

Les boucles permettent d'exécuter une série d'instructions plusieurs fois. Il existe deux types de boucles en Python : for et while.

III.1. La boucle for

La boucle for est utilisée pour itérer sur une séquence (liste, chaîne de caractères, etc.).

Syntaxe :

for élément in séquence:

instructions exécutées pour chaque élément

Exemple 1 : Calcul de la somme des n premiers nombres

```
n = int(input("Entrez la valeur de n : "))
somme = 0
for i in range(1, n+1):
    somme += i
print(f"La somme des {n} premiers nombres est {somme}.")
```

Explication : range(1, n+1) génère les nombres de 1 à n, et la variable somme accumule ces valeurs.

III.2. La boucle while

La boucle while s'exécute tant qu'une condition est vraie.

Syntaxe :

while condition:

instructions exécutées tant que la condition est vraie

Exemple 2 : Affichage des multiples de 3 jusqu'à 30

```
nombre = 3
while nombre <= 30:
    print(nombre)
```

```
nombre += 3
```

Explication :

Ce programme affiche tous les multiples de 3 en partant de 3 jusqu'à 30.

Exemples pratiques : Applications mathématiques**Exemple 1 : Calcul de la somme des carrés**

```
n = int(input("Entrez la valeur de n : "))
somme_carres = 0
for i in range(1, n+1):
    somme_carres += i**2
print(f"La somme des carrés des {n} premiers nombres est {somme_carres}.")
```

Explication :

On itère de 1 à n et on calcule le carré de chaque nombre avant de l'ajouter à la somme.

Exemple 2 : Table de multiplication

```
nombre = int(input("Entrez un nombre pour afficher sa table de multiplication : "))
for i in range(1, 11):
    print(f"{nombre} x {i} = {nombre * i}")
```

Explication :

Le programme affiche la table de multiplication du nombre donné pour les valeurs de 1 à 10.

Exemple 3 : Évaluation de séries mathématiques - Série harmonique

La **série harmonique** est une série de nombres réels. C'est la série des inverses des entiers naturels non nuls :

$$\sum_{n=1}^{\infty} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \dots$$

```
n = int(input("Entrez la valeur de n : "))
somme_harmonique = 0.0
```

```
for k in range(1, n+1):  
    somme_harmonique += 1/k  
  
print(f"La somme de la série harmonique pour n={n} est {somme_harmonique:.4f}")
```

Explication :

Ce programme évalue la somme harmonique jusqu'à l'entier n en utilisant une boucle for.

Exercices corrigés**Exercice 1 : Calcul de la somme des nombres pairs de 1 à n****Enoncé :**

Écrire un programme qui calcule la somme des nombres pairs compris entre 1 et un entier n.

Solution :

```
n = int(input("Entrez la valeur de n : "))  
  
somme_paires = 0  
  
for i in range(2, n+1, 2):  
    somme_paires += i  
  
print(f"La somme des nombres pairs jusqu'à {n} est {somme_paires}.")
```

Exercice 2 : Calculer la factorielle d'un nombre**Enoncé :**

Écrire un programme qui calcule la factorielle d'un nombre donné par l'utilisateur.

Solution :

```
n = int(input("Entrez un nombre : "))  
  
factorielle = 1  
  
for i in range(1, n+1):  
    factorielle *= i  
  
print(f"La factorielle de {n} est {factorielle}.")
```

Exercice 3 : Afficher les termes de la suite de Fibonacci jusqu'à n

Énoncé :

La suite de Fibonacci commence par 0 et 1. Chaque terme suivant est la somme des deux termes précédents. Écrire un programme qui affiche les termes de cette suite jusqu'à un nombre n.

Solution :

```
n = int(input("Entrez la valeur de n : "))  
  
a, b = 0, 1  
  
print(a, end=" ")  
  
while b <= n:  
  
    print(b, end=" ")  
  
    a, b = b, a + b
```

IV. Les structures imbriquées

Une structure imbriquée signifie qu'une structure se trouve à l'intérieur d'une autre. Lorsqu'une ou plusieurs variables membres d'une structure sont elles-mêmes des structures, on parle de structures imbriquées.

IV.1. Le if imbriqué

Il est possible d'inclure une instruction if à l'intérieur d'une autre instruction if

Exemple 1 : Cas de vérification des nombres positifs

```
nombre = int(input("Entrez un nombre : "))  
  
# instruction externe if  
  
if nombre >= 0:  
  
    ## instruction interne de type « if » (si)  
  
    if nombre == 0:  
  
        print(f"{nombre} est nulle.")  
  
    # instruction interne else
```

```
else:
```

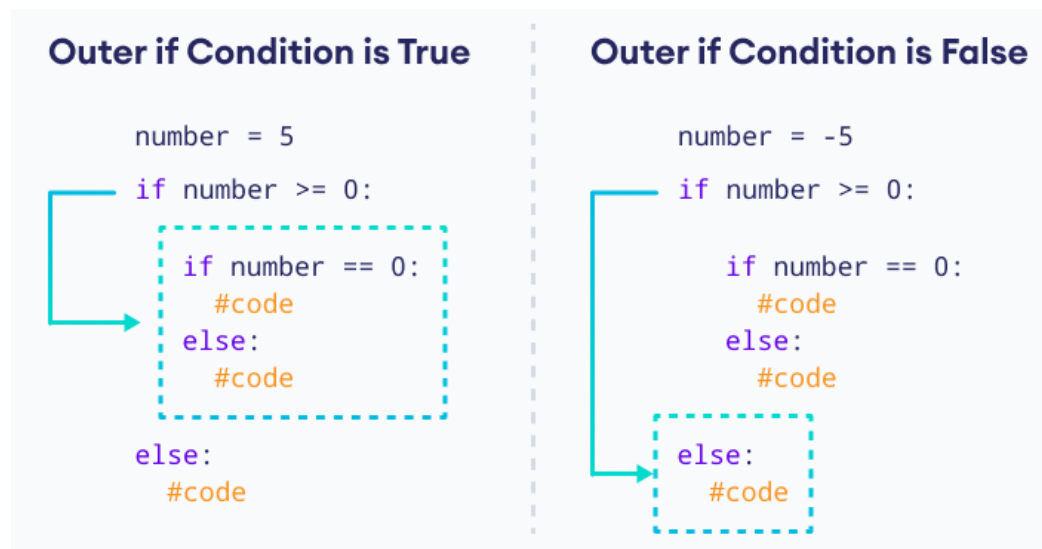
```
    print(f"{nombre} est un nombre positif.")
```

```
# instruction externe else
```

```
else:
```

```
    print(f"{nombre} est un nombre négatif.")
```

Explication :



IV.2. Les boucles imbriquées

Une boucle imbriquée signifie qu'une boucle est placée à l'intérieur d'une autre boucle. Cela permet d'effectuer des actions répétitives sur plusieurs niveaux. Les boucles imbriquées sont souvent utilisées dans des situations où l'on doit parcourir des structures à plusieurs dimensions (comme des matrices ou des tableaux 2D).

Boucle for imbriquée

Une boucle for imbriquée est souvent utilisée pour parcourir des tableaux ou des matrices. Imaginons un tableau à deux dimensions (ou une matrice). Pour parcourir chaque élément de la matrice, nous utilisons deux boucles for : la première pour parcourir les lignes, et la deuxième pour parcourir les colonnes.

Exemple : Boucle for imbriquée - Multiplication de matrices

Supposons que nous ayons deux matrices A et B, et que nous souhaitons calculer le produit de ces deux matrices.

Matrices :

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

La multiplication des matrices A x B se fait en prenant chaque élément de la première ligne de la matrice A et en multipliant par chaque élément de la colonne correspondante dans B.

```
# Matrices A et B

A = [[1, 2], [3, 4]]

B = [[5, 6], [7, 8]]

# Initialisation de la matrice résultat

resultat = [[0, 0], [0, 0]]

# Multiplication de matrices

for i in range(len(A)): # Parcourt des lignes de A

    for j in range(len(B[0])): # Parcourt des colonnes de B

        for k in range(len(B)): # Parcourt des lignes de B

            resultat[i][j] += A[i][k] * B[k][j]

# Affichage du résultat

for r in resultat:

    print(r)
```

Explication du code :

- 1) **Initialisation des matrices** : Nous avons deux matrices A et B.
- 2) **Matrice résultat** : La matrice resultat est initialisée à des zéros. Elle sera mise à jour au fur et à mesure que la multiplication sera effectuée.
- 3) **Boucles imbriquées** :

- ✓ La première boucle `for i in range(len(A))` parcourt chaque ligne de la matrice A.
- ✓ La deuxième boucle `for j in range(len(B[0]))` parcourt chaque colonne de la matrice B.
- ✓ La troisième boucle `for k in range(len(B))` effectue la multiplication entre les éléments correspondants de la matrice A et la matrice B.

4) **Affichage du résultat** : Enfin, nous parcourons la matrice resultat et affichons chaque ligne.

Boucle while imbriquée

Les boucles while imbriquées fonctionnent de manière similaire, mais elles continuent d'exécuter un bloc de code tant qu'une condition est vraie.

Exemple : Boucle while imbriquée - Création d'une table de multiplication

Dans cet exemple, nous allons utiliser des boucles while imbriquées pour créer une table de multiplication de 1×1 à 10×10

```
# Initialisation des variables

i = 1

# Première boucle while pour parcourir les lignes

while i <= 10:

    j = 1 # Réinitialisation de j pour chaque ligne

    # Deuxième boucle while pour parcourir les colonnes

    while j <= 10:

        print(f'{i * j:3}', end=' ') # Affichage avec un espace

        j += 1 # Incrémentation de j

    print() # Changer de ligne après chaque ligne de multiplication

    i += 1 # Incrémentation de i
```

Explication du code :

1. **Initialisation** : La variable `i` est initialisé à 1, ce qui représente la première ligne de la table de multiplication.
2. **Première boucle while (ligne)** : La première boucle `while` continue tant que `i` est inférieur ou égal à 10. Elle correspond à chaque ligne de la table de multiplication.
3. **Deuxième boucle while (colonne)** : La deuxième boucle `while` est imbriquée dans la première et continue tant que `j` (les colonnes) est inférieur ou égal à 10. Elle permet de calculer et afficher chaque produit.
4. **Affichage** : Le produit de `i * j` est affiché avec un formatage pour s'assurer que chaque nombre occupe trois espaces (`{i * j:3}`).
5. **Incrémentation** : À chaque itération, `j` et `i` sont incrémentés jusqu'à ce que la condition de la boucle ne soit plus vérifiée.

IV.3. Points clés à retenir :

- Les boucles imbriquées permettent de parcourir des structures multi-dimensionnelles comme des matrices.
- Dans une boucle `for` imbriquée, chaque niveau de boucle dépend de la boucle parent, ce qui permet de traiter les lignes, colonnes, etc.
- Dans une boucle `while` imbriquée, les conditions de continuation doivent être gérées avec précaution pour éviter des boucles infinies.

Exercice corrigé : Calcul de la série de Fibonacci avec des boucles imbriquées

Enoncé :

La série de Fibonacci est une séquence où chaque nombre est la somme des deux précédents, avec les deux premiers nombres étant 0 et 1. La série commence donc par : 0, 1, 1, 2, 3, 5, 8, 13, etc.

Nous allons utiliser des boucles `for` et des structures de contrôle conditionnelles (`if`, `else`) pour calculer et afficher les nombres de la série de Fibonacci jusqu'à un certain nombre `n`.

Travail à faire :

- 1) Utiliser une boucle pour répéter des calculs.
- 2) Imbriquer des structures conditionnelles pour contrôler l'exécution du code.
- 3) Afficher les résultats de manière formatée.

Correction :

```
# Fonction pour calculer la série de Fibonacci jusqu'à n termes

def fibonacci(n):

    # Liste pour stocker les termes de la série de Fibonacci

    fib_sequence = []

    # Les deux premiers termes de la série de Fibonacci

    a, b = 0, 1

    # Utilisation d'une boucle pour générer les termes de la série

    for i in range(n):

        # Ajouter le terme courant à la liste

        fib_sequence.append(a)

        # Calculer le prochain terme

        a, b = b, a + b

    return fib_sequence

# Fonction principale

def main():

    # Demander à l'utilisateur combien de termes il souhaite voir

    n = int(input("Combien de termes de la série de Fibonacci souhaitez-vous afficher ? "))

    # Vérifier si l'entrée est valide

    if n <= 0:

        print("Le nombre de termes doit être un entier positif.")

    else:

        # Calculer la série de Fibonacci

        fib_sequence = fibonacci(n)
```

```
# Afficher les termes de la série

print("Les termes de la série de Fibonacci sont :")

for i, term in enumerate(fib_sequence):

    print(f"Term {i + 1}: {term}")

# Exécuter le programme principal

if __name__ == "__main__":

    main()
```

Explication du Code

1) Définition de la Fonction fibonacci(n)

- ✓ La fonction fibonacci génère la série de Fibonacci jusqu'au terme n et retourne la liste des termes.
- ✓ On initialise deux variables a et b pour les deux premiers termes (0 et 1).
- ✓ La boucle for itère n fois, ajoutant chaque terme à la liste fib_sequence et mettant à jour les valeurs de a et b pour les termes suivants.

2) Fonction main()

- ✓ On demande à l'utilisateur combien de termes de la série il souhaite afficher.
- ✓ On vérifie que l'entrée de l'utilisateur est valide ($n > 0$). Sinon, on affiche un message d'erreur.
- ✓ On appelle la fonction fibonacci(n) pour obtenir la liste des termes et on les affiche un par un avec un index.

V. Les instruction Break et continue

Ces deux instructions sont utilisées dans les boucles (for ou while) pour contrôler leur exécution.

Imaginons que nous voulons écrire un programme qui recherche un nombre spécifique dans une liste et affiche des informations supplémentaires pendant cette recherche. Nous allons

utiliser break pour arrêter la recherche lorsque le nombre est trouvé, et continue pour ignorer certains éléments dans la liste.

Objectif du Programme

- 1) Parcourir une liste de nombres.
- 2) Afficher chaque nombre en cours de traitement.
- 3) Si le nombre est négatif, passer au nombre suivant sans afficher de message.
- 4) Si le nombre est égal à un nombre cible, afficher un message et arrêter la boucle.

Voici un programme Python illustrant ces concepts :

```
# Définition de la liste de nombres
nombres = [3, 5, -2, 7, 0, 9, -1, 12]

# Nombre cible que nous cherchons
nombre_cible = 9

# Parcourir chaque nombre dans la liste
for nombre in nombres:

    # Afficher le nombre en cours de traitement
    print(f"Traitement du nombre: {nombre}")

    # Si le nombre est négatif, passer au nombre suivant
    if nombre < 0:

        print("Nombre négatif trouvé, passage au nombre suivant...")

        continue # Passe au prochain nombre dans la liste

    # Si le nombre est égal au nombre cible, afficher un message et arrêter la boucle
    if nombre == nombre_cible:

        print(f"Nombre cible {nombre_cible} trouvé !")

        break # Arrête la boucle
```

```
# Message final après la boucle  
print("Recherche terminée.")
```

Explication du Code

1) Définition des Variables :

- ✓ nombres : Une liste de nombres à parcourir.
- ✓ nombre_cible : Le nombre que nous voulons trouver dans la liste.

2) Boucle for :

- ✓ On parcourt chaque nombre dans la liste nombres.

3) Affichage du Nombre :

- ✓ On affiche chaque nombre traité pour suivre l'exécution du programme.

4) Condition pour les Nombres Négatifs :

- ✓ Si le nombre est négatif (nombre < 0), on affiche un message indiquant que nous passons au nombre suivant et utilisons continue pour sauter le reste du code de la boucle et passer directement au prochain nombre.

5) Recherche du Nombre Cible :

- ✓ Si le nombre est égal au nombre_cible, on affiche un message que le nombre cible a été trouvé et on utilise break pour sortir de la boucle immédiatement.

6) Message Final :

- ✓ Après la fin de la boucle, que ce soit en trouvant le nombre cible ou en ayant parcouru toute la liste, on affiche un message indiquant que la recherche est terminée.

Points Clés à Retenir

- ✓ **break** : Utilisé pour arrêter immédiatement l'exécution de la boucle en cours.
- ✓ **continue** : Utilisé pour passer directement à l'itération suivante de la boucle, en sautant le reste du code dans l'itération actuelle.

Ce programme simple démontre comment utiliser break et continue de manière efficace dans des boucles pour gérer la logique conditionnelle.

VI. Conclusion

Ce cours a couvert les bases des structures de contrôle en Python, avec un accent sur les **conditions** et les **boucles**. Vous avez vu des exemples pratiques appliqués aux mathématiques comme les **séries** ou les **tables de multiplication**. Les exercices fournis vous permettront de pratiquer davantage et de comprendre ces concepts plus en profondeur.

Chapitre 3 : Les Fonctions et Modules

I. Introduction

Ce cours a pour objectif d'introduire les concepts de fonctions et de modules en Python, deux éléments fondamentaux dans la programmation structurée. Nous aborderons les points suivants :

- ✓ La définition de fonctions, avec la gestion des paramètres et des valeurs de retour.
- ✓ La portée des variables (locale vs globale).
- ✓ L'utilisation de bibliothèques standards telles que math et random.
- ✓ La création et l'importation de modules personnalisés.

Nous illustrerons ces concepts à travers des exemples concrets et des exercices pratiques.

II. Définition des Fonctions

II.1. Qu'est-ce qu'une fonction

Une fonction est un bloc de code réutilisable qui permet d'effectuer une tâche spécifique. Elle peut recevoir des **paramètres** en entrée et retourner une **valeur** en sortie.

Structure de base d'une fonction

```
def nom_de_la_fonction(param1, param2):  
  
    # bloc de code  
  
    return valeur_de_retour
```

Exemple : Fonction pour calculer la somme de deux nombres

```
def somme(a, b):
```

```
    return a + b
```

Appel de la fonction :

```
resultat = somme(3, 5)
```

```
print(resultat) # Affiche 8
```

Paramètres et Valeurs de Retour

- **Paramètres** : Variables définies entre les parenthèses lors de la définition de la fonction. Elles peuvent être utilisées à l'intérieur de la fonction.
- **Valeur de retour** : C'est le résultat produit par la fonction, renvoyé par l'instruction `return`.

Exemple concret : Fonction pour calculer une série mathématique

Prenons un exemple de fonction pour calculer la somme des n premiers nombres.

Code :

```
def somme_serie(n):
```

```
    somme = 0
```

```
    for i in range(1, n+1):
```

```
        somme += i
```

```
    return somme
```

Appel de la fonction :

```
resultat = somme_serie(5)
```

```
print(resultat) # Affiche 15 (car 1 + 2 + 3 + 4 + 5 = 15)
```

III. Portée des Variables : Locale vs Globale

Variables locales

Une variable **locale** est définie à l'intérieur d'une fonction et n'est accessible qu'au sein de cette fonction.

```
def fonction_locale():  
  
    x = 5 # Variable locale  
  
    print(x)  
  
fonction_locale() # Affiche 5  
  
# print(x) provoquera une erreur car x n'est pas définie en dehors de la fonction.
```

Variables globales

Une variable **globale** est définie en dehors des fonctions et est accessible depuis n'importe quelle partie du programme.

```
x = 10 # Variable globale  
  
def fonction_globale():  
  
    global x  
  
    x = x + 5  
  
    print(x)  
  
fonction_globale() # Affiche 15  
  
print(x) # Affiche aussi 15
```

Exemples pratiques avec des Fonctions

Fonction pour vérifier si un nombre est premier

Un nombre est dit **premier** s'il n'a que deux diviseurs : 1 et lui-même.

Code :

```
def est_premier(n):  
  
    if n < 2:  
  
        return False  
  
    for i in range(2, int(n**0.5) + 1):  
  
        if n % i == 0:  
  
            return False
```

```
return True
```

Appel de la fonction :

```
print(est_premier(17)) # Affiche True car 17 est premier  
print(est_premier(18)) # Affiche False car 18 n'est pas premier
```

IV. Modules et Bibliothèques en Python

Qu'est-ce qu'un module ?

Un module est un fichier contenant des définitions de fonctions, de variables, et de classes qui peuvent être réutilisées dans d'autres programmes. Python propose de nombreux **modules standards** comme math, random, etc.

Utilisation de la bibliothèque math

Le module math contient plusieurs fonctions mathématiques utiles, comme sqrt (racine carrée), sin, cos, etc.

Exemple : Calcul de la racine carrée

```
import math  
  
def calcul_racine_carre(nombre):  
    return math.sqrt(nombre)  
  
print(calcul_racine_carre(16)) # Affiche 4.0
```

Utilisation de la bibliothèque random

Le module random permet de générer des nombres aléatoires, très utile pour des simulations ou des tests statistiques.

Exemple : Générer un nombre aléatoire

```
import random  
  
def nombre_aleatoire(min, max):  
    return random.randint(min, max)  
  
print(nombre_aleatoire(1, 10)) # Affiche un nombre aléatoire entre 1 et 10
```

V. Création et Importation de Modules Personnalisés

Comment créer un module ?

Un module Python est simplement un fichier .py contenant des fonctions. Vous pouvez créer un module, par exemple mon_module.py, et y définir des fonctions.

Exemple : Créer un fichier mon_module.py

```
# Contenu du fichier mon_module.py

def addition(a, b):

    return a + b

def soustraction(a, b):

    return a - b
```

Comment importer un module ?

Une fois le module créé, vous pouvez l'importer dans un autre fichier Python avec l'instruction import.

Exemple d'importation :

```
# Fichier principal

import mon_module

resultat_addition = mon_module.addition(10, 5)

resultat_soustraction = mon_module.soustraction(10, 5)

print(resultat_addition) # Affiche 15

print(resultat_soustraction) # Affiche 5
```

Exemple concret : Module pour résoudre des équations mathématiques simples

Nous allons créer un module appelé equation.py pour résoudre une équation quadratique de la forme $ax^2+bx+c=0$.

Code du fichier equation.py :

```
import math
```

```
def resoudre_equation(a, b, c):  
  
    delta = b**2 - 4*a*c  
  
    if delta < 0:  
  
        return "Pas de solution réelle"  
  
    elif delta == 0:  
  
        x = -b / (2*a)  
  
        return f"Une solution réelle : x = {x}"  
  
    else:  
  
        x1 = (-b + math.sqrt(delta)) / (2*a)  
  
        x2 = (-b - math.sqrt(delta)) / (2*a)  
  
        return f"Deux solutions réelles : x1 = {x1}, x2 = {x2}"
```

Exemple d'utilisation :

```
import equation  
  
print(equation.resoudre_equation(1, -3, 2)) # Affiche "Deux solutions réelles : x1 = 2.0, x2 = 1.0"  
  
print(equation.resoudre_equation(1, 2, 1)) # Affiche "Une solution réelle : x = -1.0"
```

Exercices Pratiques

Exercice 1 : Fonction pour calculer le factoriel d'un nombre

Énoncé :

Écrivez une fonction factorielle(n) qui prend en entrée un entier n et retourne le factoriel de n.

Solution:

```
def factoriel(n):  
  
    if n == 0 or n == 1:  
  
        return 1  
  
    else:
```

```
return n * factoriel(n - 1)

print(factoriel(5)) # Affiche 120
```

Exercice 2 : Création d'un module pour calculer la moyenne

Énoncé :

Créez un module statistiques.py contenant une fonction moyenne(liste) qui calcule la moyenne d'une liste de nombres.

Solution du fichier statistiques.py :

```
def moyenne(liste):

    return sum(liste) / len(liste)
```

Utilisation dans un autre fichier :

```
import statistiques

notes = [12, 15, 14, 16, 18]

print(statistiques.moyenne(notes)) # Affiche 15.0
```

VI. Conclusion

Les fonctions et les modules sont essentiels pour structurer et organiser vos programmes en Python. La réutilisation du code à travers des fonctions bien définies et la modularisation à l'aide de modules sont des pratiques clés pour améliorer l'efficacité et la maintenabilité du code. Vous êtes maintenant capable de :

- ✓ Créer des fonctions avec des paramètres et des valeurs de retour.
- ✓ Comprendre la portée des variables.
- ✓ Utiliser des bibliothèques standards comme math et random.
- ✓ Créer et importer vos propres modules.

Chapitre 4 : Les Structures de Données

Introduction

Les structures de données en Python sont des outils essentiels pour organiser, stocker et manipuler efficacement des données. Ce cours détaillé va couvrir quatre structures de données clés : les listes, les tuples, les dictionnaires, et les ensembles, ainsi que les techniques avancées comme les compréhensions de listes et de dictionnaires.

Objectifs du cours

- Comprendre les listes, tuples, dictionnaires et ensembles en Python.
- Manipuler ces structures de données à travers des exemples pratiques.
- Apprendre à utiliser les compréhensions pour générer rapidement des listes et des dictionnaires.

I. Les Listes

Une liste en Python est une collection ordonnée et modifiable. Elle permet de stocker des éléments de tout type (entiers, chaînes, objets).

I.1. Création d'une Liste

```
# Création d'une liste vide  
ma_liste = []  
  
# Liste avec différents types d'éléments  
ma_liste = [1, "bonjour", 3.14]
```

I.2. Manipulation des Listes

Ajout d'éléments

- Utilisez `append()` pour ajouter un élément à la fin de la liste.

```
ma_liste.append(42)
```

- Utilisez `insert()` pour ajouter un élément à une position spécifique.

```
ma_liste.insert(1, "élément")
```

Suppression d'éléments

- Utilisez `remove()` pour supprimer une valeur spécifique.

```
ma_liste.remove(3.14)
```

- Utilisez `pop()` pour supprimer l'élément à une position donnée (ou le dernier si aucune position n'est spécifiée).

```
ma_liste.pop() # Supprime le dernier élément
```

Tri d'une liste

```
liste_nombres = [5, 2, 9, 1]
```

```
liste_nombres.sort() # Trie la liste en place
```

1.3. Exemple Pratique : Calcul de Moyenne à partir d'une Liste

```
# Calcul de la moyenne d'une liste de notes
```

```
notes = [12, 15, 14, 18, 10]
```

```
moyenne = sum(notes) / len(notes)
```

```
print(f"La moyenne des notes est : {moyenne}")
```

II. Les Tuples

Les tuples sont similaires aux listes, mais ils sont immuables, c'est-à-dire que leurs éléments ne peuvent pas être modifiés après la création.

II.1. Création d'un Tuple

```
# Création d'un tuple
```

```
mon_tuple = (1, 2, 3)
```

II.2. Utilisation des Tuples

Les tuples sont utiles lorsque vous voulez des données immuables, par exemple pour des coordonnées géographiques :

```
coordonnees = (48.8566, 2.3522) # Latitude et Longitude de Paris
```

III. Les Dictionnaires

Un dictionnaire est une collection non ordonnée de paires clé-valeur. Les clés sont uniques et immuables (chaînes, nombres, tuples), tandis que les valeurs peuvent être de tout type.

III.1. Création d'un Dictionnaire

```
# Création d'un dictionnaire
```

```
etudiant = {"nom": "Alice", "âge": 21, "note": 15}
```

III.2. Manipulation des Dictionnaires

Ajout ou modification d'une paire clé-valeur

```
etudiant["âge"] = 22 # Modifie l'âge
```

```
etudiant["ville"] = "Paris" # Ajoute une nouvelle clé-valeur
```

Suppression d'une paire clé-valeur

```
del etudiant["ville"]
```

III.3. Exemple Pratique : Statistiques d'étudiants

```
# Dictionnaire des statistiques d'étudiants
```

```
statistiques_etudiants = {
```

```
    "Alice": {"maths": 15, "physique": 14},
```

```
    "Bob": {"maths": 12, "physique": 16},
```

```
}
```

```
# Calcul de la moyenne de Bob en maths et physique
```

```
moyenne_bob = (statistiques_etudiants["Bob"]["maths"] +  
statistiques_etudiants["Bob"]["physique"]) / 2
```

```
print(f'Moyenne de Bob : {moyenne_bob}')
```

IV. Les Ensembles

Les ensembles sont des collections non ordonnées d'éléments uniques.

IV.1. Création et Manipulation d'un Ensemble

```
# Création d'un ensemble  
  
mon_ensemble = {1, 2, 3, 4}  
  
# Ajout d'un élément  
  
mon_ensemble.add(5)  
  
# Suppression d'un élément  
  
mon_ensemble.remove(2)
```

IV.2. Opérations sur les Ensembles

Les ensembles permettent des opérations d'union, intersection, et différence :

```
A = {1, 2, 3}  
  
B = {3, 4, 5}  
  
# Union  
  
print(A | B) # {1, 2, 3, 4, 5}  
  
# Intersection  
  
print(A & B) # {3}  
  
# Différence  
  
print(A - B) # {1, 2}
```

V. Compréhensions de Listes et Dictionnaires

V.1. Compréhension de Listes

La compréhension de listes permet de générer des listes de manière concise.

```
# Liste des carrés des nombres de 1 à 5  
  
carres = [x**2 for x in range(1, 6)]  
  
print(carres) # [1, 4, 9, 16, 25]
```

V.2. Compréhension de Dictionnaires

```
# Dictionnaire des carrés des nombres de 1 à 5  
  
carres_dict = {x: x**2 for x in range(1, 6)}  
  
print(carres_dict) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

V.3. Exemple Pratique : Dictionnaire des Puissances d'un Nombre

```
n = 2  
  
puissances = {i: i**n for i in range(1, 11)}  
  
print(puissances) # {1: 1, 2: 4, ..., 10: 100}
```

VI. Exercices corrigés

VI.1. Exercice 1 : Manipulation de Listes

- I. Créez une liste des notes de cinq étudiants.
- II. Ajoutez une note supplémentaire à la liste.
- III. Supprimez la première note et calculez la moyenne des autres.

VI.2. Exercice 2 : Statistiques avec Dictionnaire

1. Créez un dictionnaire avec trois étudiants et leurs notes en mathématiques et en physique.
2. Modifiez la note d'un étudiant.
3. Calculez la moyenne générale de chaque étudiant.

VI.3. Exercice 3 : Compréhensions

1. Utilisez une compréhension pour créer une liste des cubes des nombres de 1 à 10.
2. Créez un dictionnaire où les clés sont des nombres de 1 à 5 et les valeurs sont leurs carrés.

VII. Corrigé des Exercices

VII.1. Exercice 1 :

```
notes = [12, 15, 14, 18, 10]
```

```
notes.append(13)

notes.pop(0)

moyenne = sum(notes) / len(notes)

print(f"La moyenne est : {moyenne}")
```

VII.2. Exercice 2 :

```
etudiants = {

    "Alice": {"maths": 15, "physique": 14},

    "Bob": {"maths": 12, "physique": 16},

    "Claire": {"maths": 18, "physique": 19},

}

etudiants["Bob"]["maths"] = 14

for etudiant, notes in etudiants.items():

    moyenne = sum(notes.values()) / len(notes)

    print(f"Moyenne de {etudiant} : {moyenne}")
```

VII.3. Exercice 3 :

```
python

Copier le code

# Liste des cubes

cubes = [x**3 for x in range(1, 11)]

print(cubes)

# Dictionnaire des carrés

carres = {x: x**2 for x in range(1, 6)}

print(carres)
```

VIII. Conclusion

Les listes, tuples, dictionnaires et ensembles sont des structures de données essentielles en Python. Chaque structure a ses propres caractéristiques et est adaptée à différents types de problèmes. Les compréhensions de listes et de dictionnaires permettent de simplifier et d'accélérer la création de ces structures.

Ce cours a couvert les concepts de base et avancés sur les structures de données en Python, avec des exemples pratiques et des exercices pour bien comprendre et maîtriser ces concepts.

Chapitre 5 : Programmation Orientée Objet

I. Introduction à la Programmation Orientée Objet

I.1. Qu'est-ce que la Programmation Orientée Objet ?

La Programmation Orientée Objet (POO) est un paradigme de programmation qui permet de structurer le code en regroupant les **données** et les **comportements** associés dans des **objets**. Ces objets sont des instances de **classes**.

I.2. Concepts clés de la POO :

- ✓ **Classe** : Modèle ou plan qui définit les propriétés (attributs) et les comportements (méthodes) d'un objet.
- ✓ **Objet** : Instance concrète d'une classe.
- ✓ **Méthodes** : Fonctions définies dans une classe pour manipuler les objets.
- ✓ **Constructeur** : Méthode spéciale utilisée pour initialiser un objet lors de sa création.
- ✓ **Héritage** : Permet à une classe d'hériter les attributs et méthodes d'une autre classe.
- ✓ **Encapsulation** : Masque les détails internes d'un objet, rendant certaines informations privées.
- ✓ **Polymorphisme** : Permet aux objets de différentes classes de répondre à une même méthode de manière spécifique.

II. Les Classes et Objets en Python

II.1. Définir une Classe et Créer des Objets

En Python, une classe est définie à l'aide du mot-clé `class`. Voici un exemple simple d'une classe pour modéliser un point géométrique :

```
# Définition d'une classe Point

class Point:

    # Constructeur pour initialiser les coordonnées du point

    def __init__(self, x=0, y=0):

        self.x = x

        self.y = y

    # Méthode pour afficher les coordonnées du point

    def afficher_point(self):

        print(f"Point({self.x}, {self.y})")

# Création d'objets (instances de la classe Point)

p1 = Point(2, 3)

p2 = Point()

# Affichage des points

p1.afficher_point() # Affiche: Point(2, 3)

p2.afficher_point() # Affiche: Point(0, 0)
```

Dans cet exemple :

- ✓ La classe `Point` a un constructeur `__init__` qui initialise les coordonnées `x` et `y`.
- ✓ La méthode `afficher_point` affiche les coordonnées du point.

II.2. Héritage

L'héritage permet de créer une nouvelle classe à partir d'une classe existante. Voyons comment cela fonctionne avec un exemple.

Exemple : Héritage entre classes

```
# Classe mère : Point

class Point:

    def __init__(self, x=0, y=0):

        self.x = x

        self.y = y

    def afficher_point(self):

        print(f"Point({self.x}, {self.y})")

# Classe fille : PointColoré qui hérite de Point

class PointColoré(Point):

    def __init__(self, x=0, y=0, couleur="noir"):

        # Appel au constructeur de la classe mère

        super().__init__(x, y)

        self.couleur = couleur

    def afficher_couleur(self):

        print(f"Couleur: {self.couleur}")

# Création d'un objet PointColoré

p_color = PointColoré(4, 5, "rouge")

# Utilisation des méthodes héritées et spécifiques

p_color.afficher_point()    # Affiche: Point(4, 5)

p_color.afficher_couleur()  # Affiche: Couleur: rouge
```

Ici, PointColoré hérite de Point, donc il peut utiliser les méthodes et attributs de Point. On utilise `super()` pour appeler le constructeur de la classe mère.

II.3.Encapsulation en Python

L'encapsulation permet de protéger les attributs d'une classe pour empêcher leur modification directe.

Exemple d'encapsulation :

```
class Rectangle:

    def __init__(self, longueur, largeur):

        self.__longueur = longueur # Attribut privé

        self.__largeur = largeur # Attribut privé

    def aire(self):

        return self.__longueur * self.__largeur

    def set_longueur(self, longueur):

        if longueur > 0:

            self.__longueur = longueur

# Création d'un rectangle

rect = Rectangle(4, 2)

print(rect.aire()) # Affiche: 8

# Tentative d'accès à l'attribut privé

# rect.__longueur -> Provoque une erreur AttributeError
```

Dans cet exemple, les attributs __longueur et __largeur sont privés et ne peuvent pas être modifiés directement en dehors de la classe.

II.4.Polymorphisme en Python

Le polymorphisme permet de définir des méthodes qui peuvent avoir un comportement différent selon les objets qui les appellent.

Exemple :

```
class Forme:
```

```
def aire(self):  
    pass  
  
class Rectangle(Forme):  
    def __init__(self, longueur, largeur):  
        self.longueur = longueur  
        self.largeur = largeur  
    def aire(self):  
        return self.longueur * self.largeur  
  
class Cercle(Forme):  
    def __init__(self, rayon):  
        self.rayon = rayon  
    def aire(self):  
        return 3.14 * self.rayon * self.rayon  
  
# Utilisation polymorphique  
  
formes = [Rectangle(4, 5), Cercle(3)]  
  
for forme in formes:  
    print(forme.aire()) # Affiche 20 pour le rectangle et 28.26 pour le cercle
```

Ici, les classes Rectangle et Cercle redéfinissent la méthode aire de la classe mère Forme, mais elles ont chacune leur propre implémentation.

III. Exemples Pratiques en Mathématiques

III.1. Classe pour Modéliser un Vecteur

Voici un exemple de modélisation d'un vecteur en 2D avec les opérations de base comme l'addition et le produit scalaire.

```
class Vecteur:  
    def __init__(self, x=0, y=0):
```

```
self.x = x

self.y = y

def __add__(self, autre):

    return Vecteur(self.x + autre.x, self.y + autre.y)

def produit_scalaire(self, autre):

    return self.x * autre.x + self.y * autre.y

def afficher(self):

    print(f"Vecteur({self.x}, {self.y})")

# Création de deux vecteurs

v1 = Vecteur(1, 2)

v2 = Vecteur(3, 4)

# Addition de vecteurs

v3 = v1 + v2

v3.afficher() # Affiche: Vecteur(4, 6)

# Produit scalaire

print(v1.produit_scalaire(v2)) # Affiche: 11
```

Modélisons un vecteur mathématique en 2D avec des opérations comme l'addition et la soustraction.

```
class Vector:

    def __init__(self, x, y):

        self.x = x

        self.y = y

    def __add__(self, autre):

        return Vector(self.x + autre.x, self.y + autre.y)
```

```
def __sub__(self, autre):  
    return Vector(self.x - autre.x, self.y - autre.y)  
  
def afficher(self):  
    print(f"Vector: ({self.x}, {self.y})")  
  
# Création et manipulation de vecteurs  
  
v1 = Vector(2, 3)  
v2 = Vector(1, 5)  
v3 = v1 + v2 # Addition de vecteurs  
v3.afficher()  
  
v4 = v1 - v2 # Soustraction de vecteurs  
v4.afficher()
```

III.2. Classe pour Modéliser une Matrice

Vous pouvez aussi modéliser des matrices pour effectuer des opérations telles que l'addition ou la multiplication.

```
class Matrice:  
  
    def __init__(self, data):  
        self.data = data  
  
    def afficher(self):  
        for ligne in self.data:  
            print(ligne)  
  
    def __add__(self, autre):  
        resultat = []  
        for i in range(len(self.data)):   
            ligne = []  
            for j in range(len(self.data[0])):
```

```
        ligne.append(self.data[i][j] + autre.data[i][j])

    resultat.append(ligne)

    return Matrice(resultat)

# Création de deux matrices
m1 = Matrice([[1, 2], [3, 4]])
m2 = Matrice([[5, 6], [7, 8]])

# Addition de matrices
m3 = m1 + m2

m3.afficher() # Affiche: [6, 8], [10, 12]
```

Créez une classe Matrice pour représenter une matrice mathématique avec des opérations de base (addition, soustraction de matrices).

```
class Matrice:

    def __init__(self, matrice):

        self.matrice = matrice

    def afficher(self):

        for ligne in self.matrice:

            print(ligne)

    def __add__(self, autre):

        result = []

        for i in range(len(self.matrice)):

            ligne = [self.matrice[i][j] + autre.matrice[i][j] for j in range(len(self.matrice[0]))]

            result.append(ligne)

        return Matrice(result)

# Exemple d'utilisation
```

```
mat1 = Matrice([[1, 2], [3, 4]])  
mat2 = Matrice([[5, 6], [7, 8]])  
mat3 = mat1 + mat2  
mat3.afficher()
```

III.3. Système de gestion d'étudiants

Créez une classe Etudiant qui contient le nom et les notes d'un étudiant. Ajoutez des méthodes pour calculer la moyenne et afficher les informations de l'étudiant.

```
class Etudiant:  
  
    def __init__(self, nom, notes):  
  
        self.nom = nom  
  
        self.notes = notes  
  
    def moyenne(self):  
  
        return sum(self.notes) / len(self.notes)  
  
    def afficher(self):  
  
        print(f"Étudiant: {self.nom}")  
  
        print(f"Moyenne: {self.moyenne():.2f}")  
  
# Exemple d'utilisation  
  
etudiant1 = Etudiant("Alice", [12, 15, 14, 16])  
  
etudiant1.afficher()
```

Chapitre 6 : Interface Graphique avec Tkinter

Présentation de Tkinter

Qu'est-ce que Tkinter ?

Tkinter est une bibliothèque standard de Python pour la création d'interfaces graphiques. Elle est simple à utiliser et permet de créer des fenêtres, des boutons, des labels, des zones de texte, et bien plus encore. Tkinter est une interface vers la boîte à outils graphique Tk.

Pour utiliser Tkinter dans votre programme Python, il suffit d'importer le module :

```
import tkinter as tk
```

Création de fenêtres simples

La première étape pour créer une interface graphique avec Tkinter est de créer une fenêtre principale, appelée "fenêtre racine". Voici un exemple simple de création d'une fenêtre vide.

Exemple :

```
import tkinter as tk

# Création de la fenêtre principale

fenetre = tk.Tk()

# Titre de la fenêtre

fenetre.title("Ma première fenêtre")

# Taille de la fenêtre (largeur x hauteur)

fenetre.geometry("400x300")

# Lancement de la boucle principale

fenetre.mainloop()
```

Explication :

- ✓ `tk.Tk()` : Crée une fenêtre principale.
- ✓ `title()` : Définit le titre de la fenêtre.
- ✓ `geometry()` : Définit les dimensions de la fenêtre.

- ✓ `mainloop()` : Lancement de la boucle d'événements Tkinter, qui attend que l'utilisateur interagisse avec la fenêtre.

Ajout de Widgets

Tkinter propose plusieurs types de widgets (éléments d'interface), tels que les boutons, labels, zones de texte, etc. Voici comment ajouter certains des widgets les plus courants.

Les Labels

Les labels sont des éléments permettant d'afficher du texte.

Exemple :

```
import tkinter as tk

fenetre = tk.Tk()

# Création d'un label

label = tk.Label(fenetre, text="Bienvenue dans Tkinter !", font=("Arial", 16))

# Placement du label dans la fenêtre

label.pack()

fenetre.mainloop()
```

Les Boutons

Les boutons permettent à l'utilisateur d'interagir avec l'interface.

Exemple :

```
import tkinter as tk

fenetre = tk.Tk()

def cliquer():

    print("Le bouton a été cliqué !")

# Création d'un bouton

bouton = tk.Button(fenetre, text="Cliquez-moi", command=cliquer)

# Placement du bouton dans la fenêtre
```

```
bouton.pack()  
  
fenetre.mainloop()
```

Explication :

- ✓ `command=cliquer` : Lorsqu'on clique sur le bouton, la fonction `cliquer()` est appelée.

Les zones de texte

Les zones de texte permettent à l'utilisateur de saisir des informations.

Exemple :

```
import tkinter as tk  
  
fenetre = tk.Tk()  
  
def cliquer():  
    print("Le bouton a été cliqué !")  
  
# Création d'un bouton  
  
bouton = tk.Button(fenetre, text="Cliquez-moi", command=cliquer)  
  
# Placement du bouton dans la fenêtre  
  
bouton.pack()  
  
fenetre.mainloop()
```

Application : Interface de saisie et affichage de texte

L'exemple suivant crée une interface où l'utilisateur peut saisir du texte, puis l'afficher dans une zone spécifique après avoir appuyé sur un bouton.

Exemple :

```
import tkinter as tk  
  
fenetre = tk.Tk()  
  
fenetre.title("Saisie et affichage de texte")  
  
# Fonction pour afficher le texte saisi  
  
def afficher():
```

```
texte = zone_texte.get() # Récupérer le texte saisi

label_resultat.config(text=texte) # Modifier le texte du label pour afficher le résultat

# Création d'une zone de texte pour la saisie

zone_texte = tk.Entry(fenetre)

# Création d'un bouton pour déclencher l'affichage

bouton_afficher = tk.Button(fenetre, text="Afficher", command=afficher)

# Création d'un label pour afficher le texte saisi

label_resultat = tk.Label(fenetre, text="")

# Placement des widgets dans la fenêtre

zone_texte.pack()

bouton_afficher.pack()

label_resultat.pack()

fenetre.mainloop()
```

Explication :

- ✓ `zone_texte.get()` : Récupère le texte saisi par l'utilisateur.
- ✓ `label_resultat.config(text=texte)` : Change le texte affiché par le label pour refléter la saisie.

Gestion des événements***Introduction aux événements***

Dans Tkinter, un événement est une action réalisée par l'utilisateur, telle qu'un clic de souris, une saisie clavier, ou un survol de la souris. Chaque événement peut être associé à une fonction (ou méthode), qu'on appelle callback.

Gestion des clics de souris

On peut capturer un clic de souris et déclencher une fonction en liant cet événement avec une méthode via `bind()`.

Exemple :

```
import tkinter as tk

fenetre = tk.Tk()

def clic(event):

    print(f"Clic à la position ({event.x}, {event.y})")

# Lier un clic gauche de souris à la fonction clic()

fenetre.bind("<Button-1>", clic)

fenetre.mainloop()
```

Explication :

bind("<Button-1>", clic) : L'événement <Button-1> représente un clic gauche de souris. La fonction clic() est déclenchée à chaque clic.

Gestion de la saisie clavier

De la même manière, on peut capturer une touche du clavier.

Exemple :

```
import tkinter as tk

fenetre = tk.Tk()

def appuyer_touche(event):

    print(f"Touche appuyée : {event.char}")

# Lier une touche du clavier à la fonction appuyer_touche()

fenetre.bind("<Key>", appuyer_touche)

fenetre.mainloop()
```

Application

Application 1 : Création d'une calculatrice simple

Nous allons créer une calculatrice simple avec des boutons pour les chiffres et les opérations de base (addition, soustraction, multiplication, division). Chaque bouton, lorsqu'il est cliqué, déclenchera une action associée à l'opération ou à l'ajout d'un chiffre.

Correction :

```
import tkinter as tk

fenetre = tk.Tk()

fenetre.title("Calculatrice")

expression = ""

# Fonction pour mettre à jour l'expression en fonction du bouton cliqué
def ajouter_valeur(valeur):

    global expression

    expression += str(valeur)

    affichage.config(text=expression)

# Fonction pour évaluer l'expression mathématique
def evaluer():

    global expression

    try:

        resultat = str(eval(expression)) # Calculer l'expression

        affichage.config(text=resultat)

        expression = "" # Réinitialiser après le calcul

    except:

        affichage.config(text="Erreur")

        expression = ""
```

```
# Fonction pour réinitialiser l'affichage

def clear():

    global expression

    expression = ""

    affichage.config(text="")

# Création de l'affichage

affichage = tk.Label(fenetre, text="", font=("Arial", 20), bg="white", width=24, height=2)

affichage.grid(row=0, column=0, columnspan=4)

# Liste des boutons (chiffres et opérations)

boutons = [

    ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),

    ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),

    ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),

    ('0', 4, 0), ('.', 4, 1), ('+', 4, 3),

    ('=', 4, 2), ('C', 5, 0)

]

# Création et placement des boutons

for (texte, ligne, colonne) in boutons:

    if texte == "=":

        bouton = tk.Button(fenetre, text=texte, width=10, height=2, command=evaluer)

    elif texte == "C":

        bouton = tk.Button(fenetre, text=texte, width=10, height=2, command=clear)

    else:
```

```
bouton = tk.Button(fenetre, text=texte, width=10, height=2, command=lambda t=texte:
ajouter_valeur(t))

bouton.grid(row=ligne, column=colonne)

fenetre.mainloop()
```

Application 2 : Création d'une Interface pour Saisir des Informations et les Afficher dans une Zone de Texte

Créons maintenant un exemple pratique où l'utilisateur saisit son nom et son âge, puis ces informations sont affichées dans une zone de texte.

Correction :

```
import tkinter as tk

def afficher_info():

    nom = entree_nom.get()

    age = entree_age.get()

    zone_texte.delete(1.0, tk.END) # Effacer la zone de texte

    zone_texte.insert(tk.END, f"Nom: {nom}\nÂge: {age} ans")

# Fenêtre principale

root = tk.Tk()

root.title("Formulaire d'information")

root.geometry("400x300")

# Label et zone de texte pour le nom

label_nom = tk.Label(root, text="Nom:")

label_nom.pack()

entree_nom = tk.Entry(root)

entree_nom.pack()

# Label et zone de texte pour l'âge
```

```
label_age = tk.Label(root, text="Âge:")  
label_age.pack()  
entree_age = tk.Entry(root)  
entree_age.pack()  
  
# Bouton pour afficher les informations  
bouton_afficher = tk.Button(root, text="Afficher", command=afficher_info)  
bouton_afficher.pack()  
  
# Zone de texte pour afficher les informations  
zone_texte = tk.Text(root, height=5, width=30)  
zone_texte.pack()  
  
root.mainloop()
```

Application 3 : dire bonjour dans une zone de texte

Créer une interface qui demande à l'utilisateur de saisir son nom dans une zone de texte et d'afficher "Bonjour, [nom]" lorsqu'il appuie sur un bouton.

Correction :

```
import tkinter as tk # Importer la bibliothèque Tkinter  
  
# Fonction qui sera appelée lorsque l'utilisateur clique sur le bouton  
  
def dire_bonjour():  
    nom = entree_nom.get() # Récupérer le texte saisi dans la zone de texte  
    message_bonjour.config(text=f"Bonjour, {nom}") # Mettre à jour le label avec le message  
    de bienvenue  
  
# Créer la fenêtre principale  
  
fenetre = tk.Tk() # Initialiser la fenêtre principale  
  
fenetre.title("Salutation") # Donner un titre à la fenêtre  
  
# Créer un label pour demander à l'utilisateur d'entrer son nom
```

```
label_nom = tk.Label(fenetre, text="Entrez votre nom :")

label_nom.pack(pady=10) # Ajouter une marge verticale de 10 pixels

# Créer une zone de saisie (Entry) pour permettre à l'utilisateur d'entrer son nom

entree_nom = tk.Entry(fenetre)

entree_nom.pack(pady=5) # Ajouter une petite marge

# Créer un bouton qui appelle la fonction dire_bonjour() lorsqu'on clique dessus

bouton = tk.Button(fenetre, text="Dire Bonjour", command=dire_bonjour)

bouton.pack(pady=10) # Ajouter une marge verticale

# Créer un label vide qui sera mis à jour pour afficher "Bonjour, [nom]"

message_bonjour = tk.Label(fenetre, text="")

message_bonjour.pack(pady=20) # Ajouter une marge verticale

# Démarrer la boucle principale de l'interface Tkinter

fenetre.mainloop()
```

Application 4 : menu contextuel grâce à un clic droit

Ecrire un programme qui permet à l'utilisateur de faire un clic droit dans une fenêtre pour faire apparaître un menu contextuel qui contient :

- ✓ Sélectionne "Option 1" ou "Option 2" pour voir le message correspondant dans la console.
- ✓ "Quitter" ferme la fenêtre.

Correction :

```
import tkinter as tk

from tkinter import Menu

# Fonction appelée lors du clic droit

def afficher_menu_contextuel(event):

    # Afficher le menu contextuel à la position du curseur de la souris
```

```
menu_contextuel.post(event.x_root, event.y_root)

# Fonction pour un élément du menu

def option1():

    print("Option 1 sélectionnée")

def option2():

    print("Option 2 sélectionnée")

# Créer la fenêtre principale

root = tk.Tk()

root.title("Clic droit avec Tkinter")

root.geometry("400x300")

# Créer un widget cadre où le clic droit sera détecté

cadre = tk.Frame(root, width=400, height=300, bg="lightblue")

cadre.pack(fill=tk.BOTH, expand=True)

# Créer un menu contextuel (menu qui apparaît au clic droit)

menu_contextuel = Menu(root, tearoff=0)

menu_contextuel.add_command(label="Option 1", command=option1)

menu_contextuel.add_command(label="Option 2", command=option2)

menu_contextuel.add_separator()

menu_contextuel.add_command(label="Quitter", command=root.quit)

# Associer le clic droit (Button-3) avec l'événement pour afficher le menu

cadre.bind("<Button-3>", afficher_menu_contextuel)

# Lancer la boucle principale de Tkinter

root.mainloop()
```